# MLOps & LLMOps

The Complete Guide — tracking, registry, deployment, serving, CI/CD, testing, monitoring, drift, A/B testing, retraining, orchestration, cloud, LLMOps, security and responsible AI

**Inside:** 23 chapters in 7 parts following your training modules • principles & maturity • MLflow tracking (code) • model registry, aliases & DVC • deployment strategies (blue-green, canary, shadow, A/B, bandits) • FastAPI ML service (full code, tests, Docker) • TensorFlow Serving + Triton/KServe/vLLM • CI/CD with GitHub Actions & quality gates • automated testing (data, model, behavioural) • monitoring & observability • drift detection with PSI/KS worked numbers • A/B testing with sample-size maths • retraining strategies • Airflow, Kubeflow, feature stores • SageMaker, Azure ML, Vertex AI • LLMOps, LLM cost optimisation (computed example), LLM monitoring & tracing • security, OWASP LLM Top 10, governance & regulation • fairness & explainability (SHAP, LIME) • design patterns & checklist • ~70 interview Q&A • cheat sheet

Prepared for Mansavi Aggarwal

# Contents


**1. Introduction to MLOps — Principles and Practices** **7**

1.1 Why MLOps? . . . 7
1.2 DevOps vs MLOps . . . 8
1.3 The ML lifecycle . . . 8
1.4 Core principles . . . 8
1.5 MLOps maturity levels (Google) . . . 9
1.6 Roles . . . 9
1.7 Tool landscape . . . 9
1.8 Reference architecture . . . 10

**2. Experiment Tracking with MLflow** **11**

2.1 Why track? . . . 11
2.2 MLflow components . . . 11
2.3 Logging API . . . 12
2.4 Comparing and querying runs . . . 13
2.5 MLflow Models: flavors, signatures, pyfunc . . . 13
2.6 MLflow for GenAI . . . 13

**3. Model Registry and Versioning** **14**

3.1 Why a registry? . . . 14
3.2 Registry workflow . . . 14
3.3 Versioning everything . . . 15
3.4 Model cards . . . 15

**4. Model Deployment Strategies** **17**

4.1 Serving patterns . . . 17
4.2 Release strategies . . . 17
4.3 Rollback and safety . . . 18
4.4 Packaging and infrastructure . . . 18
4.5 Inference optimisation . . . 19
4.6 Choosing a strategy . . . 19

**5. Building ML APIs with FastAPI** **20**

5.1 Why FastAPI . . . 20
5.2 Complete service . . . 20
5.3 Key design decisions . . . 21
5.4 Testing . . . 22
5.5 Containerise . . . 22

**6. Model Serving with TensorFlow Serving (and other model servers) 23**

6.1 What is TensorFlow Serving? . . . 23
6.2 Export a SavedModel . . . 23
6.3 Run and call . . . 24
6.4 Multiple models, versions and rollback . . . 24
6.5 Other model servers . . . 24
6.6 FastAPI vs a model server . . . 25

**7. CI/CD for Machine Learning 27**

7.1 CI, CD and CT . . . 27
7.2 GitHub Actions example . . . 27
7.3 Quality gates . . . 28
7.4 Environments, IaC and GitOps . . . 28
7.5 What makes CI/CD for ML different . . . 29

**8. Automated Testing for ML Models 30**

8.1 Unit tests for code . . . 30
8.2 Data validation . . . 30
8.3 Model tests . . . 31
8.4 Robustness, fairness and integration tests . . . 31

**9. Model Monitoring and Observability 32**

9.1 Concepts . . . 32
9.2 What to log . . . 32
9.3 SLIs, SLOs and alerts . . . 33
9.4 Incident response for models . . . 33

**10. Data Drift and Model Drift Detection 35**

10.1 Types of drift . . . 35
10.2 Statistical tests . . . 36
10.3 Worked example (computed) . . . 36
10.4 Detecting concept drift . . . 37
10.5 Practical guidance . . . 37

**11. A/B Testing for ML Models 38**

11.1 Why A/B test models? . . . 38
11.2 Design . . . 38
11.3 Sample size and significance . . . 38
11.4 Pitfalls . . . 39
11.5 Alternatives . . . 39

**12. Model Retraining Strategies** **40**

12.1 Triggers . . . 40
12.2 What data to train on . . . 40
12.3 How to retrain . . . 41
12.4 Champion–challenger and safe promotion . . . 41
12.5 Risks . . . 41

**13. ML Pipeline Orchestration** **43**

13.1 Pipelines as DAGs . . . 43
13.2 Apache Airflow . . . 43
13.3 Kubeflow Pipelines (KFP) . . . 44
13.4 Comparison . . . 44
13.5 Feature stores . . . 45

**14. Cloud MLOps — AWS SageMaker** **46**

**15. Cloud MLOps — Azure Machine Learning and GCP Vertex AI** **48**

15.1 Azure Machine Learning . . . 48
15.2 Google Cloud Vertex AI . . . 49
15.3 Choosing a cloud platform . . . 50

**16. LLMOps — MLOps for Large Language Models** **52**

16.1 What changes with LLMs . . . 52
16.2 Model selection . . . 53
16.3 Prompt management . . . 53
16.4 Evaluation . . . 53
16.5 Guardrails . . . 54
16.6 Deployment: API vs self-hosted . . . 54
16.7 The LLMOps lifecycle . . . 54

**17. Cost Optimisation for LLM Applications** **55**

17.1 Token economics . . . 55
17.2 Levers . . . 55
17.3 Self-hosting economics . . . 56
17.4 Cost observability . . . 56

**18. Monitoring LLM Applications** **57**

18.1 Tracing . . . 57
18.2 What to monitor . . . 57
18.3 Online evaluation and feedback loop . . . 58

18.4 Alerts . . . 58

**19. Security and Governance in MLOps 60**

19.1 Threats across the ML lifecycle . . . 60
19.2 Core security practices . . . 60
19.3 Adversarial ML . . . 61
19.4 LLM-specific threats . . . 61
19.5 Privacy . . . 61
19.6 Governance . . . 62

**20. Responsible AI and Model Explainability 63**

20.1 Principles . . . 63
20.2 Fairness . . . 63
20.3 Explainability . . . 64
20.4 Tools . . . 66

**21. MLOps Best Practices and Design Patterns 67**

21.1 Best practices by stage . . . 67
21.2 Design patterns . . . 67
21.3 Anti-patterns . . . 68
21.4 End-to-end reference project (putting the book together) . . . 68
21.5 Production readiness checklist . . . 69

**22. Interview Questions & Answers 70**

**23. Cheat Sheet 76**

23.1 Formulas . . . 76
23.2 Commands . . . 76
23.3 Which tool when . . . 77

# Part I — Foundations, Tracking & Registry

MLOps principles and practices, experiment tracking with MLflow, model registry and versioning

CHAPTER 1

# Introduction to MLOps — Principles and Practices

Why ML projects fail in production • what MLOps is • DevOps vs MLOps • the ML lifecycle • hidden technical debt • core principles • maturity levels • roles • tool landscape • a reference architecture

## 1.1 Why MLOps?

Most ML models never reach production, and many that do quietly degrade. The model is the easy part; the hard parts are reliable data pipelines, reproducible training, safe deployment, monitoring and retraining. **MLOps** (Machine Learning Operations) is the set of practices, culture and tools that take ML from notebooks to production and keep it healthy there — DevOps principles extended to **code + data + models**.

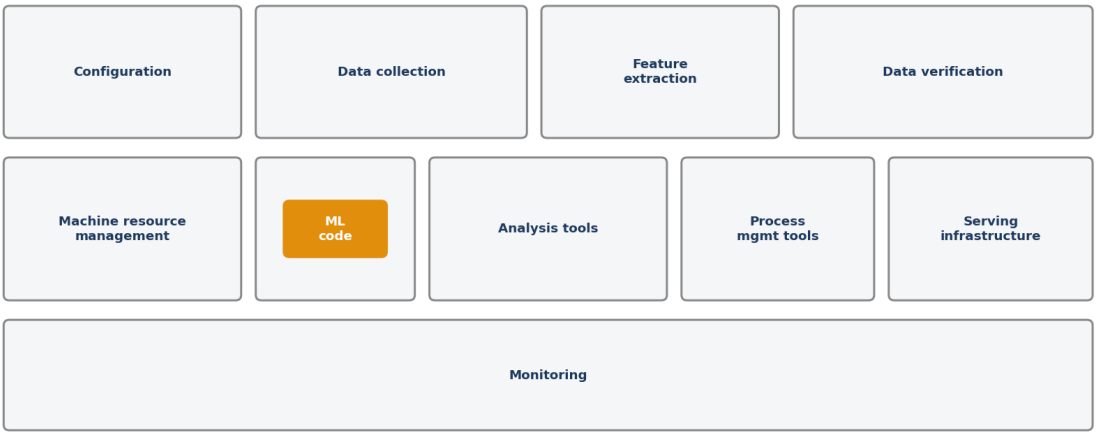



*Figure: After 'Hidden Technical Debt in ML Systems' (Sculley et al., 2015): the ML code is a small box among the infrastructure it needs.*

| Typical failure | Root cause | MLOps practice that prevents it |
|---|---|---|
| 'It worked in my notebook' | unpinned environment, manual steps | containers, pipelines, reproducibility |
| Can't reproduce last month's model | no record of data/params/code | experiment tracking + data versioning + registry |
| Production accuracy is far below offline | training–serving skew, leakage | shared feature code / feature store, validation tests |
| Model silently degrades | data/concept drift | monitoring + alerts + retraining |
| Release breaks the app | no tests, big-bang deploy | CI/CD, canary/shadow, rollback via registry |
| Nobody knows which model is live | no governance | registry, model cards, audit logs |

## 1.2 DevOps vs MLOps

| | DevOps | MLOps |
|---|---|---|
| Artefacts | code, config | code + **data** + **model** + config |
| Testing | unit, integration | + data validation, model quality gates, fairness, drift |
| Behaviour | deterministic | statistical; can degrade without any code change |
| Pipelines | CI/CD | CI/CD + **CT** (continuous training) |
| Monitoring | uptime, latency, errors | + data drift, prediction drift, model accuracy, business KPIs |
| Team | dev + ops | data scientists, ML engineers, data engineers, platform/ops, domain experts, risk/compliance |

## 1.3 The ML lifecycle

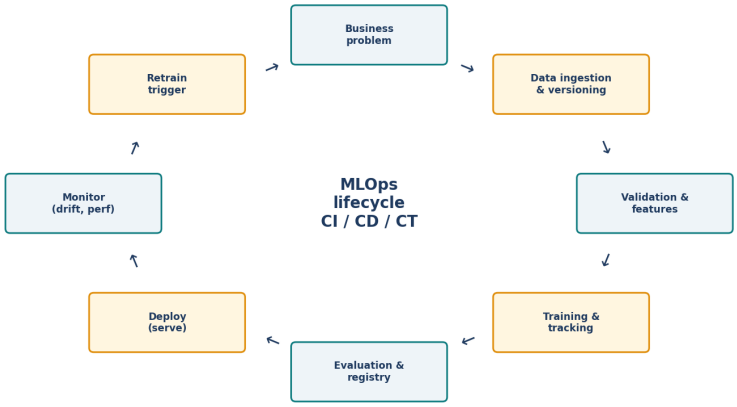



*Figure: The MLOps lifecycle is a loop: monitoring feeds retraining.*

## 1.4 Core principles

| Principle | In practice |
|---|---|
| Version everything | Git for code; DVC/lakeFS/Delta for data; registry for models; prompts and configs too |
| Reproducibility | pinned dependencies, Docker images, fixed seeds, logged params, data snapshot IDs |
| Automation | pipelines for ingestion, training, evaluation, deployment; no manual hand-offs |
| Continuous X | CI (test code+data+model), CD (release), CT (retrain), CM (monitor) |
| Testing at every layer | unit, data, model, integration, load (Chapter 8) |
| Monitoring & feedback | system + data + model + business metrics; alert and act |
| Modularity | separate data, training, serving components with clear contracts |
| Governance & responsibility | lineage, approvals, model cards, fairness, explainability, security |
| Collaboration | shared tracking server, registry and conventions across teams |

## 1.5 MLOps maturity levels (Google)

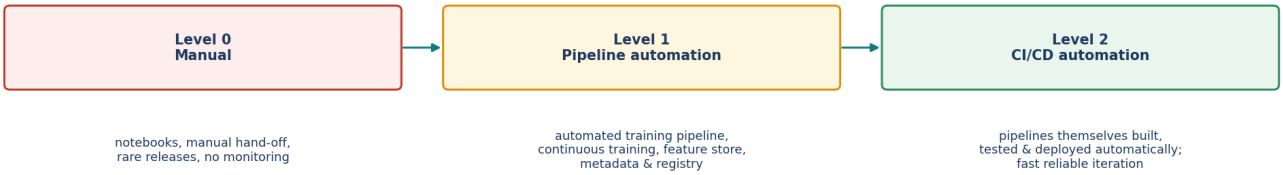



*Figure: From manual to fully automated ML.*

Microsoft describes a similar 5-level model (0: no MLOps → 4: full automated retraining and deployment). Most organisations sit at level 0–1; the goal is not the highest level everywhere but the right level for each model's value and risk.

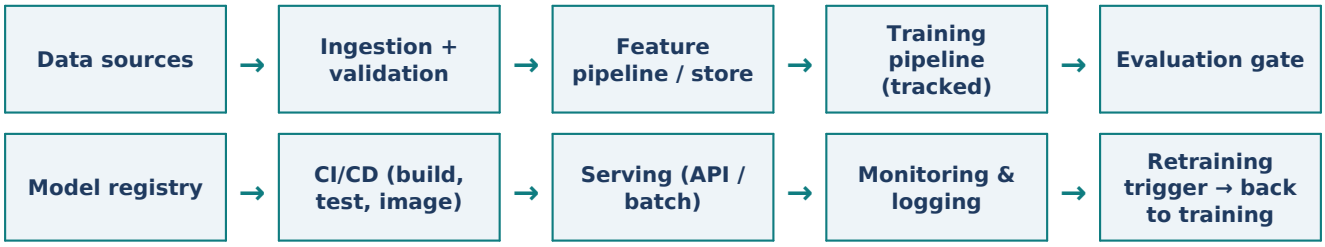
## 1.6 Roles

| Role | Focus |
|---|---|
| Data scientist | problem framing, experimentation, model quality |
| ML engineer | productionising: pipelines, serving, optimisation, tests |
| Data engineer | data ingestion, quality, feature pipelines |
| MLOps / platform engineer | infrastructure, CI/CD, registries, monitoring, cost |
| Product / domain owner | metrics, acceptance, risk |
| Risk, compliance, security | governance, audits, approvals |

## 1.7 Tool landscape

| Capability | Open source | Managed / cloud |
|---|---|---|
| Experiment tracking | MLflow, Weights & Biases (SaaS), Aim, Neptune | SageMaker Experiments, Azure ML (MLflow), Vertex Experiments |
| Data versioning | DVC, lakeFS, Delta Lake, Git LFS | lakehouse platforms |
| Feature store | Feast | SageMaker / Vertex / Databricks feature stores |
| Orchestration | Airflow, Kubeflow Pipelines, Prefect, Dagster, ZenML | SageMaker Pipelines, Azure ML pipelines, Vertex Pipelines |
| Serving | FastAPI, BentoML, TF Serving, TorchServe, Triton, KServe, Seldon, Ray Serve, vLLM | SageMaker endpoints, Azure online endpoints, Vertex endpoints |
| Monitoring | Evidently, WhyLogs, Prometheus + Grafana, OpenTelemetry | SageMaker Model Monitor, Azure ML monitoring, Vertex Model Monitoring, Arize |
| CI/CD | GitHub Actions, GitLab CI, Jenkins, Argo CD | Azure DevOps, AWS CodePipeline, Cloud Build |
| LLMOps | LangSmith, Langfuse, Phoenix, MLflow LLM eval, Ragas, promptfoo | Bedrock, Azure AI Foundry, Vertex AI |

## 1.8 Reference architecture



CHAPTER 2

# Experiment Tracking with MLflow

Why track experiments • MLflow components • tracking concepts (experiment, run, params, metrics, artifacts, tags) • tracking server & stores • logging API • autologging • nested runs & hyper-parameter search • comparing runs • model flavors & signatures • pyfunc • MLflow Projects • LLM tracing & evaluation

## 2.1 Why track?

A data scientist may run hundreds of experiments: different features, algorithms, hyper-parameters, data versions. Without tracking you cannot answer 'which settings produced the model in production?' or 'why was v3 better?'. Experiment tracking records, for every run: **parameters, metrics, artifacts, code version, data version, environment and who ran it**.

## 2.2 MLflow components

| Component | Purpose |
|---|---|
| Tracking | log and query runs (params, metrics, artifacts); UI for comparison |
| Models | standard packaging ('flavors') + signature + environment so any model can be loaded and served the same way |
| Model Registry | central store of model versions with aliases, tags, lineage, approvals (Chapter 3) |
| Projects | package code with its environment (MLproject file) for reproducible runs |
| Evaluation & tracing (LLMs) | mlflow.evaluate, LLM judges, mlflow.tracing for GenAI apps |

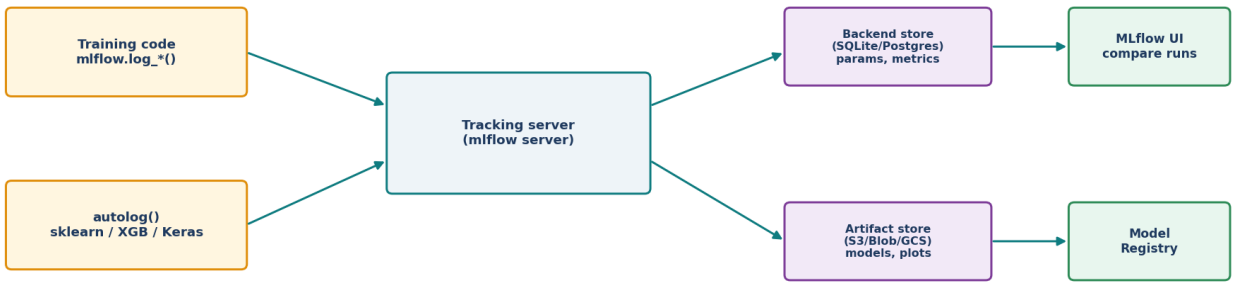



*Figure: MLflow tracking architecture.*

| Concept | Meaning |
|---|---|
| Experiment | named group of runs for one problem (e.g. 'churn') |
| Run | one execution of training code |
| Parameters | inputs (learning_rate, max_depth) — logged once |
| Metrics | numbers that can change over steps (loss per epoch, AUC) |
| Artifacts | files: model, plots, confusion matrix, feature importance, data samples |

| Concept | Meaning |
|---|---|
| Tags | metadata: git commit, author, dataset version, purpose |
| Backend store | params/metrics/metadata (file, SQLite, Postgres, MySQL) |
| Artifact store | large files (local, S3, Azure Blob, GCS) |

**running MLflow**

```
pip install mlflow
mlflow ui                                                    # local UI at http://127.0.0.1:5000 (uses ./mlruns)
mlflow server --backend-store-uri postgresql://user:pw@host/mlflow \
              --artifacts-destination s3://my-bucket/mlflow --host 0.0.0.0 --port 5000   # team server
```

# 2.3 Logging API

**track_run.py**

```
import mlflow, mlflow.sklearn
from mlflow.models import infer_signature
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, f1_score, ConfusionMatrixDisplay

mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("churn")

params = {"n_estimators": 400, "max_depth": 8, "min_samples_leaf": 3}
with mlflow.start_run(run_name="rf_baseline") as run:
    mlflow.set_tags({"data_version": "v2024-10-01", "git_sha": GIT_SHA, "owner": "mansavi"})
    mlflow.log_params(params)
    model = RandomForestClassifier(**params, random_state=42).fit(X_train, y_train)
    p = model.predict_proba(X_val)[:, 1]
    mlflow.log_metrics({"val_auc": roc_auc_score(y_val, p), "val_f1": f1_score(y_val, p > 0.5)})
    ConfusionMatrixDisplay.from_predictions(y_val, p > 0.5).figure_.savefig("cm.png");
        mlflow.log_artifact("cm.png")
    mlflow.log_dict({"features": list(X_train.columns)}, "features.json")
    signature = infer_signature(X_val, model.predict_proba(X_val))          # input/output schema
    mlflow.sklearn.log_model(model, name="model", signature=signature, input_example=X_val.head(3))
    print(run.info.run_id)
# older MLflow versions use artifact_path="model" instead of name="model"
```

## Autologging

**autolog.py**

```
mlflow.autolog()                # or mlflow.sklearn.autolog(), mlflow.xgboost.autolog(), mlflow.tensorflow.autolog()
model.fit(X_train, y_train)     # params, training metrics, model and (for sklearn) best CV params are logged automatically
```

## Logging per-epoch metrics and nested runs

**nested_runs.py**

```
with mlflow.start_run(run_name="keras_cnn"):
    for epoch in range(20):
        hist = model.fit(..., epochs=1, verbose=0)
        mlflow.log_metric("val_loss", hist.history["val_loss"][0], step=epoch)    # curve in the UI

with mlflow.start_run(run_name="optuna_search"):                                  # parent
    def objective(trial):
        with mlflow.start_run(nested=True):                                       # one child per trial
            params = {"learning_rate": trial.suggest_float("lr", 1e-3, 0.3, log=True), "max_depth":
        trial.suggest_int("max_depth", 3, 10)}
            score = cross_val_score(XGBClassifier(**params), X, y, cv=5, scoring="roc_auc").mean()
            mlflow.log_params(params); mlflow.log_metric("cv_auc", score); return score
    study = optuna.create_study(direction="maximize"); study.optimize(objective, n_trials=30)
    mlflow.log_params(study.best_params); mlflow.log_metric("best_cv_auc", study.best_value)
```

## 2.4 Comparing and querying runs

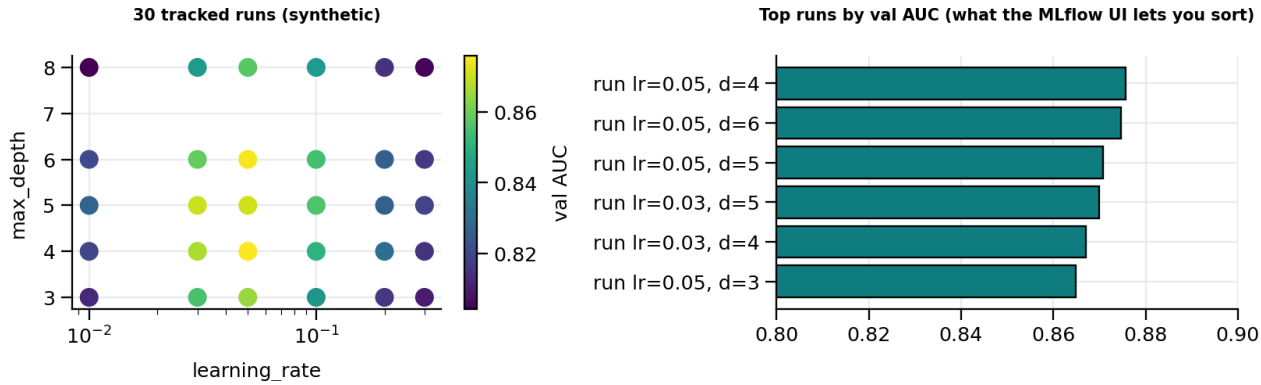



*Figure: What tracking gives you: every configuration and its score in one place (synthetic runs).*

**query_runs.py**

```
runs = mlflow.search_runs(experiment_names=["churn"], filter_string="metrics.val_auc > 0.85 and params.max_depth = '8'",
                          order_by=["metrics.val_auc DESC"], max_results=10)          # pandas DataFrame
best_run_id = runs.iloc[0]["run_id"]
model = mlflow.pyfunc.load_model(f"runs:/{best_run_id}/model")                       # load any flavor through pyfunc
model.predict(X_test)
```

## 2.5 MLflow Models: flavors, signatures, pyfunc

- A logged model is a directory with an **MLmodel** file listing its flavors (sklearn, xgboost, tensorflow/keras, pytorch, transformers, langchain, onnx…) plus **conda.yaml/requirements.txt**, **signature** (input/output schema) and optional input example.
- **python_function (pyfunc)** is the universal flavor: every model can be loaded with mlflow.pyfunc.load_model and called with .predict(DataFrame) — serving code doesn't care which library trained it.
- Custom logic (preprocessing + model + post-processing) → subclass mlflow.pyfunc.PythonModel.
- Serve locally: *mlflow models serve -m models:/churn-classifier@champion -p 5001* → REST /invocations; or build a Docker image with *mlflow models build-docker*.

**custom_pyfunc.py**

```
class ChurnWrapper(mlflow.pyfunc.PythonModel):
    def load_context(self, context): import joblib; self.pipe = joblib.load(context.artifacts["pipe"])
    def predict(self, context, model_input): return (self.pipe.predict_proba(model_input)[:, 1] >=
        0.42).astype(int)
mlflow.pyfunc.log_model(name="churn_wrapped", python_model=ChurnWrapper(), artifacts={"pipe": "pipe.joblib"})
```

## 2.6 MLflow for GenAI

Recent MLflow versions add tracing (mlflow.langchain.autolog(), @mlflow.trace), prompt versioning, and LLM evaluation (mlflow.evaluate / GenAI scorers with LLM-as-judge metrics such as relevance and faithfulness). Check your installed version's docs — this area changes quickly.

> **Interview tip:** Tracking hygiene: one experiment per problem, descriptive run names, always tag git SHA and data version, log the evaluation dataset hash, log the full pipeline (preprocessing + model), and never log secrets or raw PII as artifacts.

CHAPTER 3

# Model Registry and Versioning

What a registry is • versions, aliases, tags • lineage • promotion workflow & approvals • MLflow registry API • stages (legacy) vs aliases • versioning data with DVC • versioning code, environments, features, prompts • semantic versioning • model cards

## 3.1 Why a registry?

Tracking stores *every* run; a **model registry** stores the *candidates for production* and their lifecycle. It answers: which version is live, who approved it, what data and code produced it, and how to roll back.

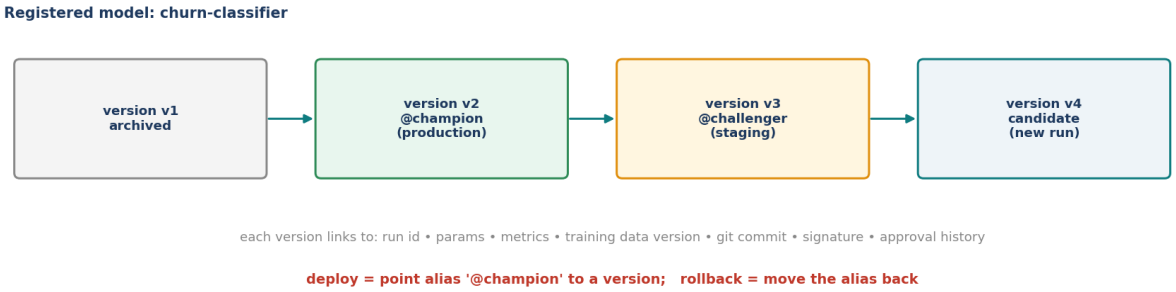
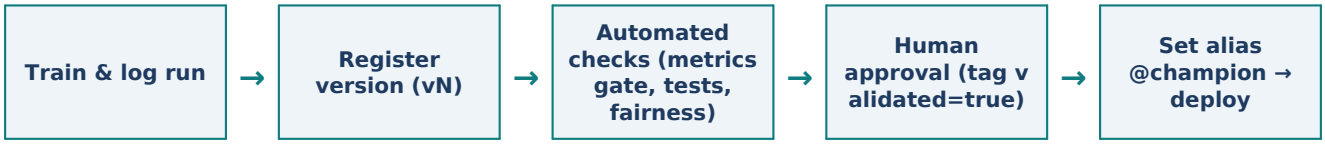



*Figure: Registered model versions with aliases.*

| Registry concept | Meaning |
|---|---|
| Registered model | a named model (e.g. churn-classifier) |
| Version | immutable numbered snapshot linked to a run (v1, v2, …) |
| Alias | movable pointer: @champion, @challenger, @staging — serving loads by alias |
| Tags / description | metadata: validation status, owner, ticket ID, model card link |
| Lineage | run → params, metrics, data version, git commit, environment |
| Stages (legacy) | None/Staging/Production/Archived — deprecated in recent MLflow in favour of aliases and tags |

## 3.2 Registry workflow



**registry.py**

```
from mlflow import MlflowClient
client = MlflowClient()
mv = mlflow.register_model(f"runs:/{run_id}/model", "churn-classifier")         # creates version N
client.set_model_version_tag("churn-classifier", mv.version, "validation_status", "passed")
client.update_model_version("churn-classifier", mv.version, description="RF, data v2024-10-01, AUC 0.86")

champion = client.get_model_version_by_alias("churn-classifier", "champion")
new_auc = client.get_run(mv.run_id).data.metrics["val_auc"]; old_auc = client.get_run(champion.run_id).data.metrics["val_auc
        "]
if new_auc > old_auc + 0.005:                                                   # promotion gate
    client.set_registered_model_alias("churn-classifier", "challenger", mv.version)   # canary first, then:
    # client.set_registered_model_alias("churn-classifier", "champion", mv.version)

model = mlflow.pyfunc.load_model("models:/churn-classifier@champion")          # serving code never hard-codes a version
# rollback = set the alias back to the previous version
```

# 3.3 Versioning everything

| What | Tool / practice |
|---|---|
| Code | Git; tag releases; commit SHA logged in every run |
| Data | DVC, lakeFS, Delta/Iceberg time-travel, immutable dated snapshots in object storage |
| Features | feature store definitions in Git; point-in-time correct training sets |
| Environment | requirements.txt / poetry lock / conda env; Docker image tag = git SHA |
| Models | registry versions + semantic version for APIs (v1, v2 routes) |
| Configs & prompts | YAML/Hydra configs in Git; prompt registry (MLflow, LangSmith hub) |
| Pipelines | pipeline definitions in Git; run IDs recorded |

## Data versioning with DVC

**dvc**

```
pip install dvc dvc-s3
git init && dvc init
dvc remote add -d storage s3://my-bucket/dvc-store
dvc add data/train.csv                 # creates data/train.csv.dvc (small pointer file with content hash)
git add data/train.csv.dvc .gitignore && git commit -m "data v1"
dvc push                               # upload the actual data to remote storage
# later: switch to an old version
git checkout <old-commit> data/train.csv.dvc && dvc checkout
# pipelines: dvc.yaml stages (prepare → train → evaluate); `dvc repro` reruns only stages whose inputs
        changed
```

# 3.4 Model cards

| Section | Content |
|---|---|
| Model details | owner, version, date, algorithm, license |
| Intended use | use cases and out-of-scope uses |
| Training data | sources, time range, preprocessing, known gaps |
| Evaluation | metrics overall and per slice; datasets |
| Fairness & ethics | group metrics, mitigations |
| Limitations & risks | known failure modes, drift sensitivity |
| Operational | latency, resources, monitoring, retraining policy, rollback |

# Part II — Deployment & Serving

Deployment strategies, building ML APIs with FastAPI, model serving with TensorFlow Serving, containers and Kubernetes

CHAPTER 4

# Model Deployment Strategies

Serving patterns (online, batch, streaming, edge) • deployment strategies (recreate, rolling, blue-green, canary, shadow, A/B, bandits) • rollback • model packaging • infrastructure (VMs, containers, Kubernetes, serverless, managed endpoints) • latency & scaling • optimisation (quantisation, ONNX, batching) • choosing a strategy

## 4.1 Serving patterns

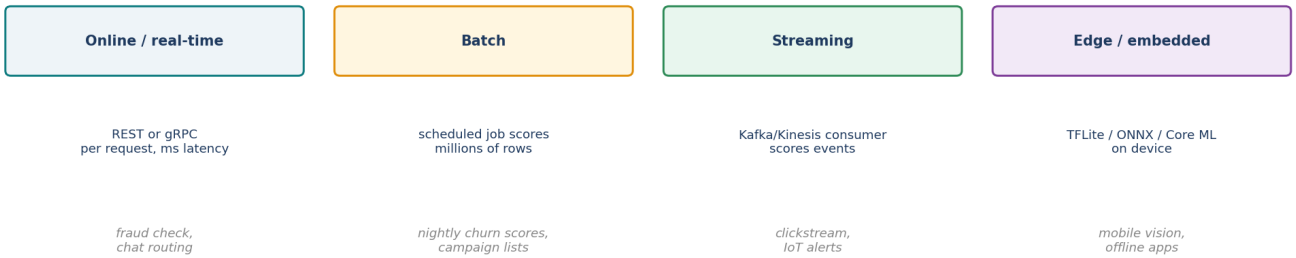



*Figure: How predictions are delivered.*

| Pattern | Latency | Cost | Freshness of inputs | Example |
|---|---|---|---|---|
| Online (sync API) | ms | always-on compute | request-time features | fraud scoring during payment |
| Asynchronous | seconds–minutes | queue + workers | request-time | document OCR, LLM batch jobs |
| Batch | hours | cheapest | as of last run | nightly propensity scores to CRM |
| Streaming | sub-second | stream infra | event-time | real-time recommendations |
| Edge | lowest, offline | device | on-device | mobile vision, keyboard prediction |

## 4.2 Release strategies

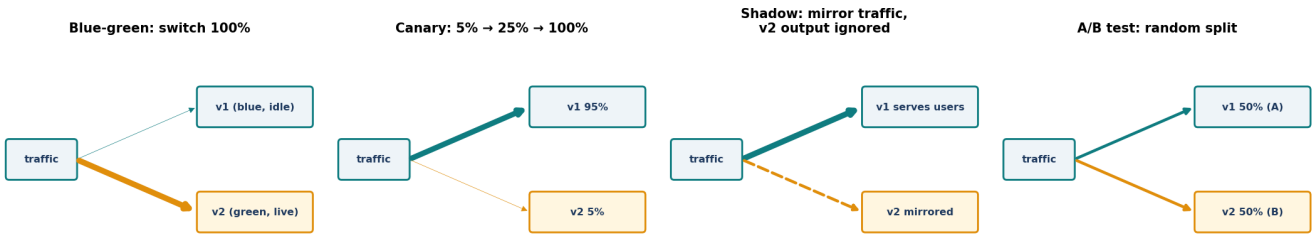



*Figure: Blue-green, canary, shadow and A/B deployments.*

| Strategy | How | Pros | Cons | Use when |
|---|---|---|---|---|
| Recreate | stop old, start new | simple | downtime | dev, batch jobs |

| Strategy | How | Pros | Cons | Use when |
|---|---|---|---|---|
| Rolling | replace instances gradually | no downtime | mixed versions temporarily | default K8s update |
| Blue-green | two full environments; switch router | instant switch & rollback | 2× infra | critical services |
| Canary | small % of live traffic → new model, ramp up if healthy | limits blast radius | needs metrics + automation | most model releases |
| Shadow (dark launch) | copy traffic to new model, ignore its output | zero user risk, real data | extra compute, no user feedback | validating big changes |
| A/B test | random user split, compare business metric | measures causal impact | needs time & traffic | choosing between models (Ch. 11) |
| Multi-armed bandit | adaptively send more traffic to the winner | less regret | harder analysis | many variants, fast feedback |

## 4.3 Rollback and safety

- Keep the previous model version warm (or deployable in minutes); rollback = move the registry alias / traffic weight back.
- Automated rollback rules: error rate, latency p95, prediction distribution shift, business guardrail metric.
- Feature flags to disable a model path and fall back to rules or the old model.
- Backward-compatible APIs: version routes (/v1, /v2) and schema validation.

## 4.4 Packaging and infrastructure

| Option | Notes |
|---|---|
| Pickle/joblib + FastAPI in Docker | simplest, flexible (Chapter 5) |
| MLflow pyfunc / BentoML | standard packaging + generated server |
| TF Serving / TorchServe / Triton | high-performance dedicated servers (Chapter 6) |
| Kubernetes (KServe, Seldon) | autoscaling, canary, multi-model; scale to zero |
| Serverless (Lambda, Cloud Run, Azure Functions) | spiky low traffic; cold starts; size limits |
| Managed endpoints (SageMaker, Azure ML, Vertex) | least ops; autoscaling, monitoring, A/B built in |

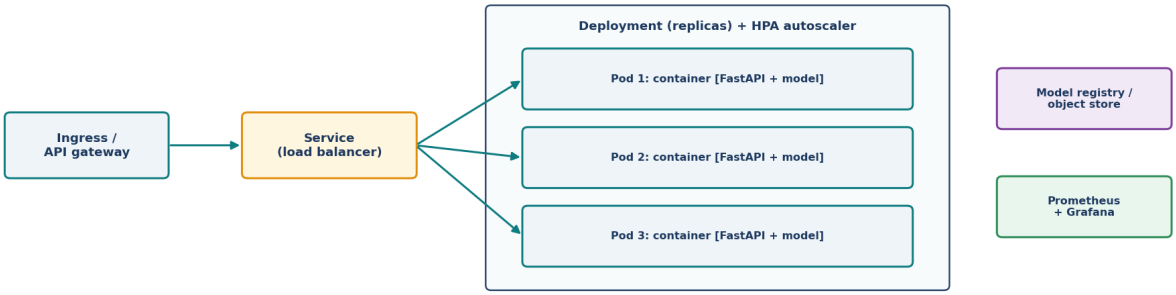



*Figure: A model API on Kubernetes: replicas behind a service, autoscaled.*

**k8s deployment + HPA**

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: churn-api}
spec:
  replicas: 3
  strategy: {type: RollingUpdate, rollingUpdate: {maxUnavailable: 0, maxSurge: 1}}
  selector: {matchLabels: {app: churn-api}}
  template:
    metadata: {labels: {app: churn-api}}
    spec:
      containers:
      - name: api
        image: myregistry.azurecr.io/churn-api:1.4.0
        ports: [{containerPort: 8000}]
        resources: {requests: {cpu: "500m", memory: "1Gi"}, limits: {cpu: "2", memory: "2Gi"}}
        readinessProbe: {httpGet: {path: /ready, port: 8000}}
        livenessProbe: {httpGet: {path: /health, port: 8000}}
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: churn-api}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: churn-api}
  minReplicas: 2
  maxReplicas: 10
  metrics: [{type: Resource, resource: {name: cpu, target: {type: Utilization, averageUtilization: 70}}}]
```

## 4.5 Inference optimisation

| Technique | Gain |
|---|---|
| Quantisation (FP16/INT8/INT4) | 2–4× smaller/faster; small accuracy loss |
| Pruning, distillation | smaller models (e.g. DistilBERT ~40% smaller) |
| ONNX Runtime / TensorRT / OpenVINO | graph optimisation, fused kernels |
| Dynamic batching | higher GPU throughput |
| Caching | repeat inputs, embeddings, features |
| Right hardware | CPU for small models, GPU for DL/LLMs; autoscale; spot for batch |

## 4.6 Choosing a strategy

- Predictions needed before the user acts? → online; otherwise batch is cheaper and simpler.
- High risk of harm/revenue loss → shadow first, then canary with automated rollback.
- Want to prove business value → A/B test.
- Small team → managed endpoints or a single FastAPI container; large platform → Kubernetes + KServe.

CHAPTER 5

# Building ML APIs with FastAPI

Why FastAPI • project layout • Pydantic schemas • loading models at startup (lifespan) • sync vs async endpoints • batch endpoints • validation & errors • versioning • middleware, logging & metrics • auth • testing • Docker • gunicorn/uvicorn workers • performance tips

## 5.1 Why FastAPI

- High performance (ASGI, Starlette, async), automatic request validation via Pydantic type hints, automatic OpenAPI docs at /docs and /redoc, dependency injection, easy testing.
- De facto standard for Python ML/LLM APIs.

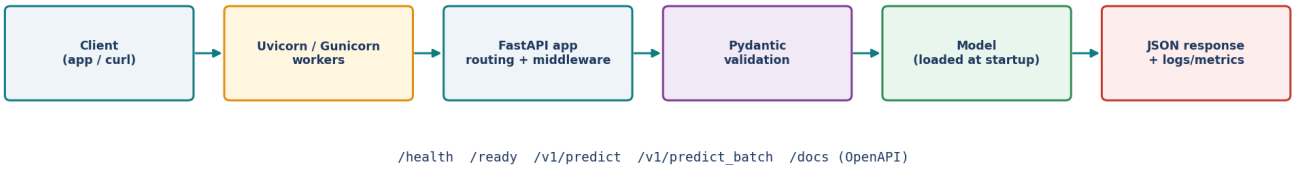



*Figure: Request path through a FastAPI model service.*

**layout**

```
churn_api/
├── app/
│   ├── main.py          # app, lifespan (model loading), routers, middleware
│   ├── schemas.py       # Pydantic request/response models
│   ├── model.py         # load_model(), predict() — wraps MLflow/joblib model
│   ├── config.py        # settings from env (pydantic-settings)
│   └── monitoring.py    # logging, Prometheus metrics
├── tests/test_api.py
├── Dockerfile  requirements.txt  .env.example
```

## 5.2 Complete service

**app/schemas.py + app/main.py**

```
# app/schemas.py
from pydantic import BaseModel, Field
from typing import Literal
class Customer(BaseModel):
    tenure: int = Field(ge=0, le=120)
    monthly_charges: float = Field(gt=0)
    contract: Literal["Month-to-month", "One year", "Two year"]
    internet_service: Literal["DSL", "Fiber optic", "No"]
class Prediction(BaseModel):
    churn_probability: float; churn: bool; model_version: str
class BatchRequest(BaseModel):
    customers: list[Customer] = Field(max_length=1000)

# app/main.py
import time, logging, uuid, pandas as pd, mlflow
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException, Depends, Request, Header
from prometheus_client import Counter, Histogram, make_asgi_app
from app.schemas import Customer, Prediction, BatchRequest
from app.config import settings

REQS = Counter("predictions_total", "predictions", ["outcome"]); LAT = Histogram("predict_latency_seconds", "latency")
STATE = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    STATE["model"] = mlflow.pyfunc.load_model(settings.model_uri)        # e.g. models:/churn-classifier@champion — loaded
        ONCE
    STATE["version"] = settings.model_version; yield; STATE.clear()

app = FastAPI(title="Churn API", version="1.0.0", lifespan=lifespan)
app.mount("/metrics", make_asgi_app())                                   # Prometheus scrape endpoint

@app.middleware("http")
async def add_request_id(request: Request, call_next):
    rid = request.headers.get("x-request-id", str(uuid.uuid4())); t = time.perf_counter()
    resp = await call_next(request); resp.headers["x-request-id"] = rid
    logging.info({"rid": rid, "path": request.url.path, "status": resp.status_code, "ms": round((time.perf_counter() - t) *
        1000, 1)})
    return resp

def verify_key(x_api_key: str = Header(...)):
    if x_api_key != settings.api_key: raise HTTPException(status_code=401, detail="invalid API key")

@app.get("/health")
def health(): return {"status": "ok"}
@app.get("/ready")
def ready(): return {"ready": "model" in STATE}

@app.post("/v1/predict", response_model=Prediction, dependencies=[Depends(verify_key)])
def predict(c: Customer):                                    # plain 'def' → runs in threadpool (CPU-bound model)
    with LAT.time():
        try: p = float(STATE["model"].predict(pd.DataFrame([c.model_dump()]))[0])
        except Exception as e: REQS.labels("error").inc(); raise HTTPException(status_code=500, detail="prediction failed")
        from e
    REQS.labels("ok").inc()
    return Prediction(churn_probability=round(p, 4), churn=p >= settings.threshold, model_version=STATE["version"])

@app.post("/v1/predict_batch", dependencies=[Depends(verify_key)])
def predict_batch(req: BatchRequest):
    probs = STATE["model"].predict(pd.DataFrame([c.model_dump() for c in req.customers]))
    return [{"churn_probability": float(p), "churn": bool(p >= settings.threshold)} for p in probs]
```

# 5.3 Key design decisions

| Topic | Guidance |
|---|---|
| Model loading | load once at startup (lifespan), never per request; warm-up call |
| def vs async def | CPU-bound inference → **def** (FastAPI runs it in a threadpool) or a process pool; I/O-bound calls (LLM APIs, DBs) → **async def** with async clients |
| Validation | Pydantic Field constraints and Literal enums reject bad inputs with 422 before the model sees them |
| Errors | HTTPException with clear status codes; never leak stack traces |
| Versioning | /v1, /v2 routes; return model_version in responses |
| Batching | batch endpoint; for GPU models use server-side dynamic batching |

| Topic | Guidance |
|---|---|
| Security | API keys/OAuth2/JWT, HTTPS at gateway, rate limiting, input size limits, CORS restrictions |
| Observability | structured JSON logs with request IDs, Prometheus metrics, OpenTelemetry traces; log inputs/predictions (minus PII) for drift |
| Workers | gunicorn -k uvicorn.workers.UvicornWorker -w (2×cores) or uvicorn --workers N; each worker holds its own model copy (memory!) |

## 5.4 Testing

**tests/test_api.py**

```
from fastapi.testclient import TestClient
from app.main import app
def test_predict_ok():
    with TestClient(app) as c:                    # 'with' triggers lifespan (model load)
        r = c.post("/v1/predict", headers={"x-api-key": "test"}, json={"tenure": 3, "monthly_charges": 89.5, "contract":
        "Month-to-month", "internet_service": "Fiber optic"})
        assert r.status_code == 200 and 0 <= r.json()["churn_probability"] <= 1
def test_validation_error():
    with TestClient(app) as c:
        r = c.post("/v1/predict", headers={"x-api-key": "test"}, json={"tenure": -1, "monthly_charges": 0, "contract":
        "Weekly", "internet_service": "DSL"})
        assert r.status_code == 422
```

## 5.5 Containerise

**Dockerfile**

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ app/
RUN useradd -m appuser
USER appuser
EXPOSE 8000
HEALTHCHECK CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"
CMD ["gunicorn", "app.main:app", "-k", "uvicorn.workers.UvicornWorker", "-w", "2", "-b", "0.0.0.0:8000"]
```

**build, run, call, load-test**

```
docker build -t churn-api:1.4.0 .
docker run -p 8000:8000 --env-file .env churn-api:1.4.0
curl -X POST localhost:8000/v1/predict -H "x-api-key: $KEY" -H "Content-Type: application/json" \
     -d '{"tenure": 3, "monthly_charges": 89.5, "contract": "Month-to-month", "internet_service": "Fiber optic"}'
locust -f loadtest.py --host http://localhost:8000      # load test: p95 latency at target RPS
```

CHAPTER 6

# Model Serving with TensorFlow Serving (and other model servers)

What TF Serving is • SavedModel format & signatures • versioned model directories • running with Docker • REST & gRPC APIs • model config & multiple models • version policies & rollback • batching • warm-up • monitoring • TorchServe, Triton, KServe, BentoML, vLLM • when to use a model server vs FastAPI

## 6.1 What is TensorFlow Serving?

A high-performance, production C++ server for TensorFlow models. It loads **SavedModel** directories, automatically discovers new numbered versions, serves them over **REST (port 8501)** and **gRPC (port 8500)**, supports request batching and hot-swapping versions without downtime.

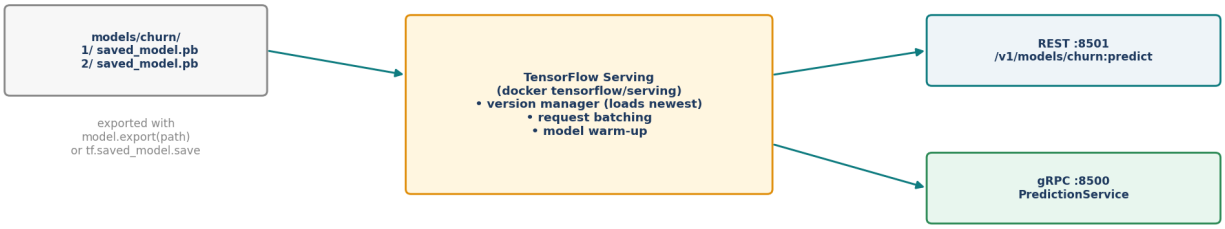



*Figure: TF Serving architecture.*

## 6.2 Export a SavedModel

**export**

```
import tensorflow as tf
model = tf.keras.models.load_model("churn_model.keras")
model.export("models/churn/1")            # Keras 3: SavedModel for serving (TF2/Keras 2: tf.saved_model.save(model,
        "models/churn/1"))
# directory layout — version numbers are integers; TF Serving serves the highest by default
# models/churn/1/saved_model.pb   models/churn/1/variables/
!saved_model_cli show --dir models/churn/1 --all          # inspect signatures: inputs, outputs, dtypes, shapes
```

**Interview tip:** Put preprocessing **inside** the exported graph (e.g. Normalization / StringLookup / TextVectorization layers) so the server receives raw features and training–serving skew is impossible.

## 6.3 Run and call

**docker + REST**

```
docker pull tensorflow/serving
docker run -d -p 8501:8501 -p 8500:8500 \
  --mount type=bind,source=$(pwd)/models/churn,target=/models/churn \
  -e MODEL_NAME=churn tensorflow/serving
curl http://localhost:8501/v1/models/churn                    # status of loaded versions
curl -X POST http://localhost:8501/v1/models/churn:predict -d '{"instances": [[3, 89.5, 1, 0, 1]]}'
curl -X POST http://localhost:8501/v1/models/churn/versions/1:predict -d '{"instances": [[3, 89.5, 1, 0, 1]]}'   # pin a
        version
```

**python client**

```
import requests, json
r = requests.post("http://localhost:8501/v1/models/churn:predict", json={"signature_name": "serving_default", "instances":
        X[:5].tolist()})
print(r.json()["predictions"])
# gRPC (lower latency, binary protobuf): tensorflow-serving-api → PredictionServiceStub, PredictRequest
```

## 6.4 Multiple models, versions and rollback

**models.config**

```
# models.config
model_config_list {
  config { name: "churn"  base_path: "/models/churn"  model_platform: "tensorflow"
           model_version_policy { specific { versions: 1 versions: 2 } }
           version_labels { key: "stable" value: 1 }  version_labels { key: "canary" value: 2 } }
  config { name: "fraud"  base_path: "/models/fraud"  model_platform: "tensorflow" }
}
# docker run ... tensorflow/serving --model_config_file=/models/models.config --model_config_file_poll_wait_seconds=60
# call by label: /v1/models/churn/labels/canary:predict  → canary routing at the client/gateway; rollback = change label
```

| Feature | How |
|---|---|
| Version policy | latest (default, N newest), all, or specific versions |
| Hot reload | drop a new numbered folder; server loads it and unloads old per policy |
| Batching | --enable_batching --batching_parameters_file (max_batch_size, batch_timeout_micros) — big GPU throughput gains |
| Warm-up | assets.extra/tf_serving_warmup_requests to avoid slow first requests |
| Monitoring | --monitoring_config_file exposes Prometheus metrics (request count, latency) |

## 6.5 Other model servers

| Server | Strengths |
|---|---|
| TorchServe | PyTorch models, handlers for custom pre/post-processing, management API |
| NVIDIA Triton Inference Server | multi-framework (TF, PyTorch, ONNX, TensorRT, Python), dynamic batching, concurrent models, GPU-optimised, ensembles |
| KServe (Kubernetes) | InferenceService CRD, autoscaling incl. scale-to-zero, canary traffic splits, multiple runtimes |
| BentoML | Python-first packaging and serving, adaptive batching, easy containers |
| MLflow models serve | serve any pyfunc model quickly |
| vLLM / TGI / Ollama | LLM serving: continuous batching, paged KV cache, OpenAI-compatible APIs |

## 6.6 FastAPI vs a model server

| Choose FastAPI when | Choose a model server when |
|---|---|
| custom business logic around the model, small/medium traffic, sklearn/XGBoost models, quick iteration, combining several calls (feature lookup + model + rules) | high-throughput deep-learning inference on GPUs, dynamic batching, many models/versions, standard REST/gRPC; often FastAPI sits in front as a gateway |

# Part III — Automation, Testing & Monitoring

CI/CD for ML, automated testing for ML models, monitoring and observability

CHAPTER 7

# CI/CD for Machine Learning

CI vs CD vs CT • what triggers what • pipeline stages • GitHub Actions example • quality gates • building & scanning images • environments & promotion • infrastructure as code • GitOps • ML-specific challenges

## 7.1 CI, CD and CT

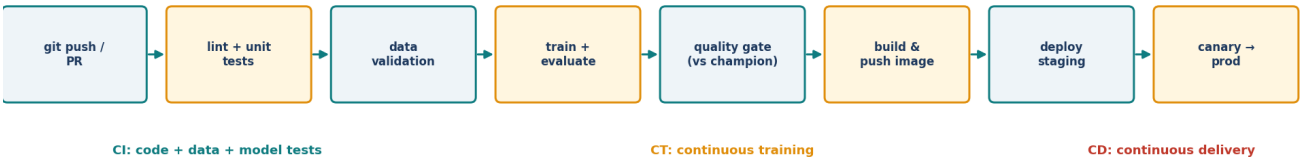



*Figure: An ML delivery pipeline: code and data tests, training, a quality gate, packaging and progressive delivery.*

| Practice | Trigger | What happens |
|---|---|---|
| Continuous Integration (CI) | push / pull request | lint, unit tests, data-schema tests, small training smoke test, build |
| Continuous Training (CT) | schedule, new data, drift alert | run training pipeline, evaluate, register candidate |
| Continuous Delivery/Deployment (CD) | model approved / merge to main | build image, deploy to staging, integration & load tests, canary to prod |
| Continuous Monitoring (CM) | always | metrics, drift, alerts → may trigger CT |

## 7.2 GitHub Actions example

**.github/workflows/ml.yml**

```
name: ml-pipeline
on:
  pull_request:
  push: {branches: [main]}
  schedule: [{cron: "0 2 * * 1"}]          # weekly retrain (CT)
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.11", cache: pip}
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: ruff check . && black --check .
      - run: pytest -q tests/unit tests/data                  # code + data tests
  train-evaluate:
    needs: ci
    if: github.event_name != 'pull_request'
    runs-on: ubuntu-latest
    env: {MLFLOW_TRACKING_URI: "${{ secrets.MLFLOW_URI }}"}
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: dvc pull                                         # versioned data
      - run: python pipelines/train.py --config configs/prod.yaml
      - run: python pipelines/evaluate.py --min-auc 0.80 --max-drop-vs-champion 0.005   # QUALITY GATE (non-zero exit fails
        job)
      - run: python pipelines/register.py --alias challenger
  deploy:
    needs: train-evaluate
    runs-on: ubuntu-latest
    environment: production                                   # requires manual approval in GitHub
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t $REGISTRY/churn-api:${{ github.sha }} . && docker push $REGISTRY/churn-api:${{ github.sha }}
      - run: trivy image $REGISTRY/churn-api:${{ github.sha }}   # vulnerability scan
      - run: kubectl set image deploy/churn-api api=$REGISTRY/churn-api:${{ github.sha }}   # or update GitOps repo / Argo
        Rollouts canary
```

## 7.3 Quality gates

| Gate | Example rule |
|---|---|
| Absolute performance | AUC ≥ 0.80, recall@precision 0.9 ≥ 0.6 |
| Relative to champion | not worse than champion by > 0.5 pp on the same test set |
| Slice performance | no segment (region, product) drops > 2 pp |
| Fairness | selection-rate ratio ≥ 0.8 across groups |
| Operational | p95 latency < 100 ms, image < 1 GB, memory < 1 GB |
| Data | schema valid, missing rate < threshold, no train/test overlap |
| Security | no critical CVEs, no secrets in image |

## 7.4 Environments, IaC and GitOps

- dev → staging → production with the **same** image promoted (build once, deploy many).
- Infrastructure as Code (Terraform, Bicep, CloudFormation) for clusters, endpoints, storage, permissions.
- GitOps (Argo CD, Flux): the desired deployment state lives in Git; merging a change deploys it; rollback = git revert.
- Progressive delivery tools: Argo Rollouts, Flagger, managed endpoint traffic splitting.

## 7.5 What makes CI/CD for ML different

- Tests are statistical — define tolerances, use fixed evaluation sets and seeds.
- Training can be slow/expensive → smoke-train on a sample in CI, full training in CT pipelines on proper compute.
- Data changes trigger pipelines, not only code changes.
- Artifacts are large → store in object storage/registry, not Git.
- Human approval often required for regulated models.

CHAPTER 8

# Automated Testing for ML Models

Test pyramid for ML • unit tests • data validation (Great Expectations, Pandera) • model performance tests • slice tests • behavioural tests (invariance, directional, minimum functionality) • fairness tests • robustness • regression tests vs champion • integration & API tests • load tests • LLM tests

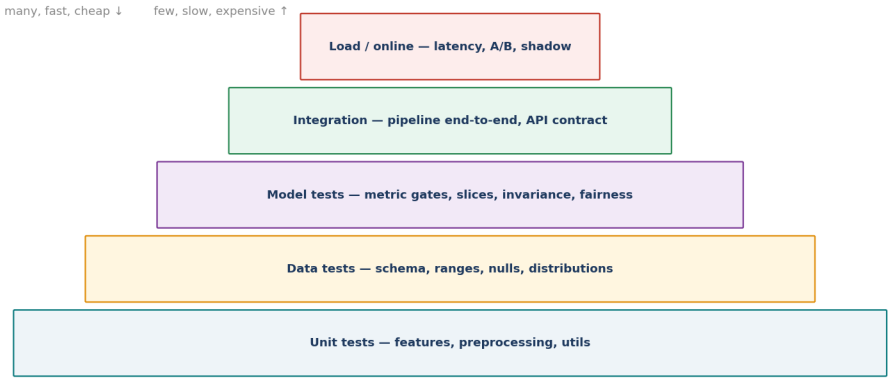



*Figure: Testing pyramid for ML systems.*

## 8.1 Unit tests for code

**unit tests**

```
# tests/unit/test_features.py
import pandas as pd, numpy as np
from src.features import add_tenure_bucket, clean_charges
def test_tenure_bucket_edges():
    df = pd.DataFrame({"tenure": [0, 12, 13, 72]})
    assert add_tenure_bucket(df)["tenure_bucket"].tolist() == ["new", "new", "mid", "loyal"]
def test_clean_charges_handles_blanks():
    s = clean_charges(pd.Series(["29.85", " ", None]))
    assert s.isna().sum() == 2 and s.iloc[0] == 29.85
def test_pipeline_output_shape(sample_df, fitted_pipeline):
    assert fitted_pipeline.predict_proba(sample_df).shape == (len(sample_df), 2)
```

## 8.2 Data validation

**data tests (Pandera)**

```
import pandera.pandas as pa                    # older versions: import pandera as pa
from pandera import Column, Check
schema = pa.DataFrameSchema({
    "tenure": Column(int, Check.in_range(0, 120)),
    "monthly_charges": Column(float, Check.gt(0)),
    "contract": Column(str, Check.isin(["Month-to-month", "One year", "Two year"])),
    "churn": Column(int, Check.isin([0, 1])),
}, checks=[Check(lambda df: df["churn"].mean().__ge__(0.05), error="churn rate implausibly low")])
schema.validate(df, lazy=True)              # raises with ALL failures listed
# Great Expectations offers the same idea with expectation suites, data docs and checkpoints in pipelines
```

| Data check | Example |
|---|---|
| Schema | columns, dtypes, allowed categories |
| Completeness | missing rate per column below threshold |
| Ranges & validity | age 0–120, amounts ≥ 0, dates not in the future |
| Uniqueness & integrity | IDs unique, foreign keys exist |
| Distribution | mean/quantiles within tolerance of reference; PSI below limit |
| Leakage | no target-derived features; no overlap between train and test IDs; time ordering |

## 8.3 Model tests

**model tests**

```
import pytest
from sklearn.metrics import roc_auc_score
def test_min_performance(model, X_test, y_test):
    assert roc_auc_score(y_test, model.predict_proba(X_test)[:, 1]) >= 0.80
def test_not_worse_than_champion(model, champion, X_test, y_test):
    new = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1]); old = roc_auc_score(y_test,
        champion.predict_proba(X_test)[:, 1])
    assert new >= old - 0.005
@pytest.mark.parametrize("segment", ["Month-to-month", "One year", "Two year"])
def test_slice_performance(model, X_test, y_test, segment):
    m = X_test["contract"] == segment
    assert roc_auc_score(y_test[m], model.predict_proba(X_test[m])[:, 1]) >= 0.70
def test_invariance_to_customer_id(model, X_test):                # behavioural: irrelevant change → same output
    X2 = X_test.copy(); X2["customer_id"] = "XXXX"
    assert (abs(model.predict_proba(X2)[:, 1] - model.predict_proba(X_test)[:, 1]) < 1e-9).all()
def test_directional_expectation(model, base_row):                # behavioural: higher charges → churn risk should not
        fall
    hi = base_row.copy(); hi["monthly_charges"] *= 1.5
    assert model.predict_proba(hi)[0, 1] >= model.predict_proba(base_row)[0, 1] - 0.01
def test_deterministic(model, X_test): assert (model.predict(X_test) == model.predict(X_test)).all()
```

| Behavioural test (CheckList) | Idea | NLP example |
|---|---|---|
| Minimum functionality | simple cases must be right | 'I love it' → positive |
| Invariance | label-preserving perturbation shouldn't change output | change names/locations, typos |
| Directional expectation | a change should move output in a known direction | add 'not' → less positive |

## 8.4 Robustness, fairness and integration tests

- Robustness: noise, missing values, unseen categories, extreme values, adversarial perturbations — model must not crash and should degrade gracefully.
- Fairness: group metrics (Chapter 20) within tolerance.
- Integration: full pipeline on a small sample; API contract tests; serialisation round-trip (save → load → same predictions).
- Load: p95 latency and error rate at target RPS (Locust, k6).
- LLM apps: golden-set evals with LLM-as-judge, regression tests on prompts, red-team suites for prompt injection and unsafe output (promptfoo, DeepEval, Ragas).

CHAPTER 9

# Model Monitoring and Observability

Monitoring vs observability • four layers of metrics • logging predictions • delayed labels & proxy metrics • dashboards & alerts • SLIs/SLOs • tools (Prometheus, Grafana, OpenTelemetry, Evidently) • incident response

## 9.1 Concepts

**Monitoring** tracks known metrics and alerts when they cross thresholds. **Observability** is the ability to understand *why* something happened from logs, metrics and traces (the three pillars), plus — for ML — logged inputs, predictions and explanations.

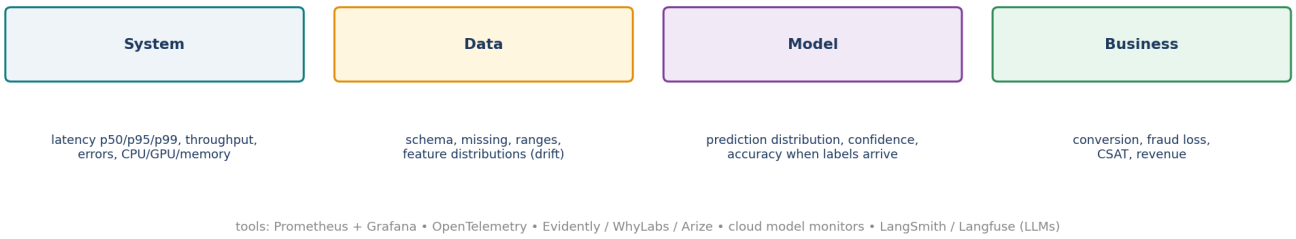



*Figure: What to monitor in an ML system.*

## 9.2 What to log

| Item | Why |
|---|---|
| request ID, timestamp, model version | trace and attribute every prediction |
| input features (or hashed/summarised; no raw PII) | drift detection, debugging, retraining data |
| prediction, score, confidence | prediction drift, threshold analysis |
| latency per stage | performance SLOs |
| ground-truth label when it arrives (join by ID) | real accuracy |
| explanations for sampled requests | audit, debugging |

### Delayed ground truth

Fraud labels may arrive weeks later; churn labels after 30 days. Meanwhile monitor **proxies**: input drift, prediction-distribution drift, confidence, business proxies (approval rate), and human-review samples.

## 9.3 SLIs, SLOs and alerts

| SLI | Example SLO | Alert |
|---|---|---|
| availability | 99.9% successful requests / 30 days | error rate > 1% for 5 min |
| latency | p95 < 150 ms | p95 > 200 ms for 10 min |
| data quality | missing rate < 2% per feature | > 5% in last hour |
| drift | PSI < 0.2 on top features | PSI > 0.25 for 2 consecutive days |
| model quality | weekly AUC ≥ 0.80 | < 0.78 → page + retrain ticket |

**evidently**

```
# Evidently (open source) — data drift + quality report against a reference set
from evidently import Report
from evidently.presets import DataDriftPreset, DataSummaryPreset
report = Report([DataDriftPreset(), DataSummaryPreset()])
result = report.run(reference_data=train_df, current_data=last_week_df)
result.save_html("drift_report.html")
# (API differs between Evidently versions — older: from evidently.report import Report; from evidently.metric_preset import
        DataDriftPreset)
```

## 9.4 Incident response for models

1. Detect (alert) → acknowledge.
2. Triage: system issue (latency, errors) vs data issue (pipeline broke, schema change) vs model issue (drift).
3. Mitigate: rollback to previous model, fall back to rules, disable feature, scale up.
4. Fix root cause: repair pipeline, retrain, add a test/monitor that would have caught it.
5. Post-mortem, update runbook.

# Part IV — Drift, Experimentation & Retraining

Data and model drift detection, A/B testing for ML models, model retraining strategies

CHAPTER 10

# Data Drift and Model Drift Detection

Types of drift • drift patterns • statistical tests (KS, chi-square, PSI, JS, Wasserstein) with formulas • worked PSI example • multivariate & embedding drift • concept-drift detection • training–serving skew • code • what to do when drift is detected

## 10.1 Types of drift

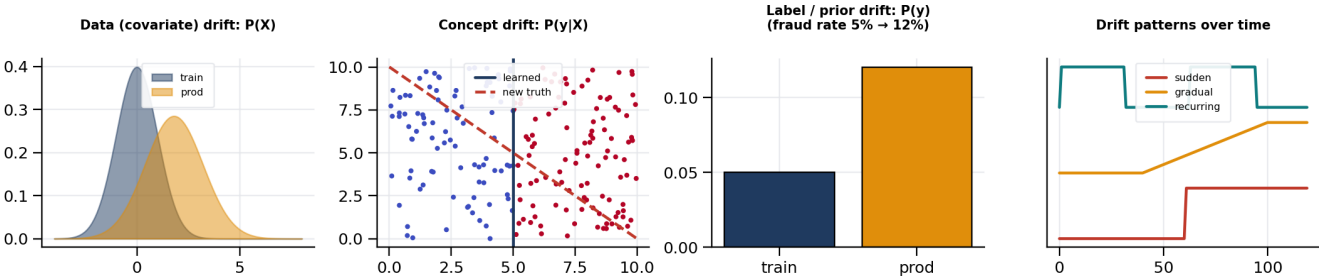



*Figure: Data drift, concept drift, label drift, and how drift unfolds over time.*

$$P_{train}(X, y) = P(y \mid X)\, P(X) \quad \neq \quad P_{prod}(X, y)$$

| Type | What changes | Example | Detect with |
|---|---|---|---|
| Data / covariate drift | P(X) | new customer segment, inflation shifts amounts, new device types | feature distribution tests (no labels needed) |
| Concept drift | P(y \| X) | fraudsters change tactics; COVID changed buying behaviour | performance on new labels, error-rate monitors (DDM, ADWIN) |
| Label / prior drift | P(y) | fraud rate doubles | label distribution, prediction distribution |
| Prediction drift | distribution of ŷ | share of 'approve' jumps | early proxy when labels are delayed |
| Training–serving skew | feature computed differently online vs offline | bug, timezone, unit change | compare logged serving features with training features |
| Upstream data change | schema/encoding/units changed | 'cm' → 'mm', new category code | schema & range validation |

## 10.2 Statistical tests

$$\mathrm{PSI} = \sum_{i=1}^{B} (a_i - e_i) \ln \frac{a_i}{e_i} \qquad D_{KS} = \sup_x |F_{ref}(x) - F_{cur}(x)| \qquad \chi^2 = \sum_k \frac{(O_k - E_k)^2}{E_k}$$

$$JS(P\|Q) = \frac{1}{2} KL(P\|M) + \frac{1}{2} KL(Q\|M),\ M = \frac{P+Q}{2} \qquad W_1(P,Q) = \int |F_P(x) - F_Q(x)|\, dx$$

| Method | Data type | Notes / typical thresholds |
|---|---|---|
| PSI (Population Stability Index) | numeric (binned) or categorical | < 0.1 stable; 0.1–0.25 moderate; > 0.25 significant (industry rule of thumb, from credit scoring) |
| Kolmogorov–Smirnov test | numeric | max CDF gap; p-value; very sensitive with large n — combine with effect size |
| Chi-square test | categorical | observed vs expected counts |
| Jensen–Shannon distance | both | symmetric, bounded [0,1] (base 2) |
| Wasserstein (earth mover's) | numeric | in units of the feature; interpretable magnitude |
| Domain classifier | multivariate | train a model to distinguish reference vs current; AUC ≫ 0.5 → drift |
| Embedding drift | text/images | distance between mean embeddings, MMD, or domain classifier on embeddings |

## 10.3 Worked example (computed)

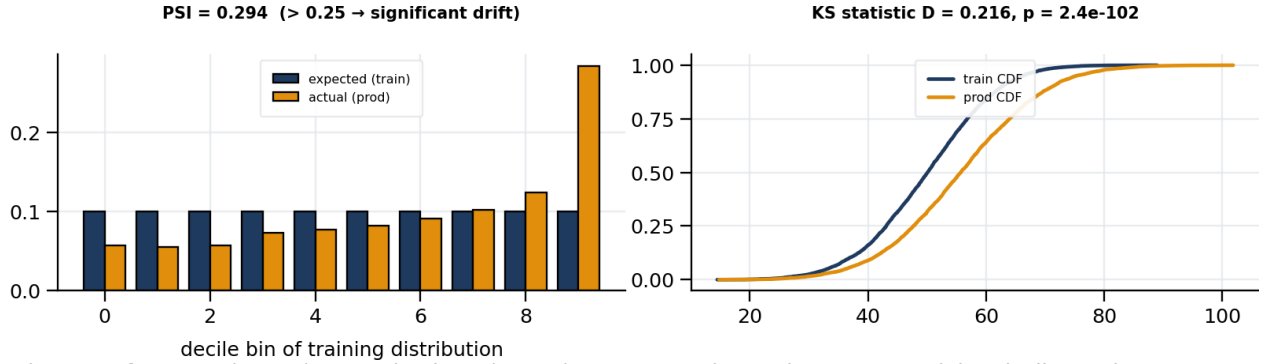



*Figure: Reference N(50, 10) vs production N(56, 12), 5,000 samples each: PSI over training deciles and KS on CDFs.*

Bins = deciles of the training distribution, so each expected share $e_i = 0.10$. Production shifts right, so upper bins grow. Result: **PSI = 0.294** (above 0.25 → significant) and **KS D = 0.216** with a tiny p-value. A single bin contributes, e.g., (0.20 − 0.10)·ln(0.20/0.10) = 0.069.

**drift_checks.py**

```
import numpy as np
from scipy.stats import ks_2samp, chi2_contingency
def psi(ref, cur, bins=10, eps=1e-6):
    edges = np.quantile(ref, np.linspace(0, 1, bins + 1)); edges[0], edges[-1] = -np.inf, np.inf
    e = np.histogram(ref, edges)[0] / len(ref) + eps; a = np.histogram(cur, edges)[0] / len(cur) + eps
    return float(np.sum((a - e) * np.log(a / e)))
report = {}
for col in numeric_cols:
    report[col] = {"psi": psi(train[col], prod[col]), "ks_p": ks_2samp(train[col], prod[col]).pvalue}
for col in categorical_cols:
    table = pd.crosstab(pd.concat([train[col], prod[col]]), ["ref"] * len(train) + ["cur"] * len(prod))
    report[col] = {"chi2_p": chi2_contingency(table)[1]}
drifted = [c for c, r in report.items() if r.get("psi", 0) > 0.25 or r.get("chi2_p", 1) < 0.01]
```

## 10.4 Detecting concept drift

- When labels arrive: rolling-window accuracy/AUC/calibration vs baseline; alert on sustained drops.
- Streaming detectors on the error stream: DDM, EDDM, ADWIN, Page-Hinkley (river library).
- Without labels: prediction drift + confidence drop + human-review samples + business KPIs.
- Feature-importance shift (SHAP distributions) can reveal changed relationships.

## 10.5 Practical guidance

- Monitor the most important features first (by permutation/SHAP importance); drift in unimportant features rarely matters.
- Use windows (daily/weekly) and require persistence (e.g. 2 consecutive windows) to avoid alert fatigue.
- Large samples make every tiny shift 'significant' — rely on effect sizes (PSI, Wasserstein) not only p-values.
- Seasonality is not drift: compare with the same period last year or a seasonal reference.
- Response playbook: investigate (pipeline bug? real change?) → if real and performance is affected, retrain on recent data / reweight → validate → canary.

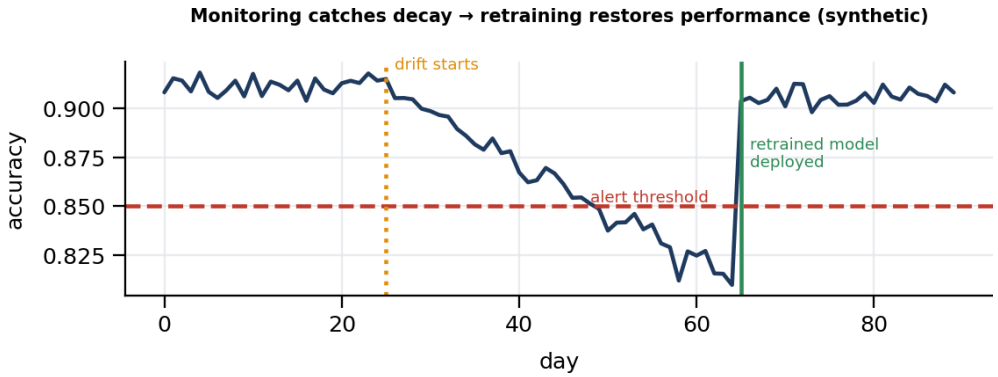



*Figure: Synthetic timeline: drift degrades accuracy; alert; retrained model restores it.*

CHAPTER 11

# A/B Testing for ML Models

Why offline metrics aren't enough • experiment design • hypotheses & metrics (primary, guardrail) • randomisation unit • sample size & power (worked) • significance tests • common pitfalls (peeking, novelty, SRM, interference) • interleaving • multi-armed bandits • implementation

## 11.1 Why A/B test models?

A model with better offline AUC may not improve the business: offline data reflects the old model's decisions, metrics may not capture user behaviour, and latency matters. An **A/B test** (online controlled experiment) randomly splits users between the current model (A, control) and the new model (B, treatment) and measures the causal effect on business metrics.

## 11.2 Design

1. Hypothesis: 'New recommender increases click-through rate by ≥ 1 percentage point.'
2. Metrics: **primary** (CTR or conversion), **guardrails** (latency, revenue per user, complaints, fairness), **diagnostic** (prediction distribution).
3. Randomisation unit: user (consistent experience) — hash(user_id) % 100 < 50 → B; avoid per-request randomisation for user-facing features.
4. Sample size from baseline rate, minimum detectable effect (MDE), α (0.05) and power (0.8).
5. Run for full business cycles (≥ 1–2 weeks), no early stopping on peeks.
6. Analyse with the pre-registered test; check sample-ratio mismatch; decide; document.

## 11.3 Sample size and significance

$$n \approx \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 [p_A(1-p_A) + p_B(1-p_B)]}{(p_B - p_A)^2} \qquad z = \frac{\hat{p}_B - \hat{p}_A}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_A} + \frac{1}{n_B}\right)}}$$

**Worked**: baseline 10%, detect +1 pp (11%), α = 0.05 (z = 1.96), power 0.8 (z = 0.84): n ≈ $(2.80)^2 \cdot (0.09 + 0.0979)/0.01^2$ ≈ **14,748 users per group**. Halving the MDE quadruples the sample size.

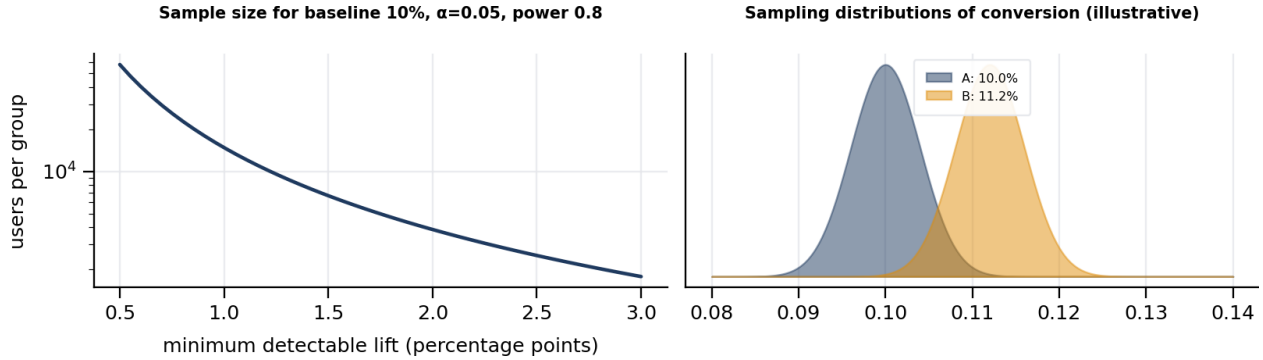



*Figure: Left: required sample size vs minimum detectable lift (computed). Right: overlapping sampling distributions of two conversion rates.*

**ab_analysis.py**

```
from statsmodels.stats.proportion import proportions_ztest, proportion_confint
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize
n = NormalIndPower().solve_power(effect_size=proportion_effectsize(0.11, 0.10), alpha=0.05, power=0.8)   # per
        group
stat, p = proportions_ztest(count=[conv_B, conv_A], nobs=[n_B, n_A], alternative="larger")
ci_B = proportion_confint(conv_B, n_B, method="wilson")
# continuous metrics (revenue/user): Welch t-test or bootstrap CIs; many metrics → correct for multiple testing
```

## 11.4 Pitfalls

| Pitfall | Fix |
|---|---|
| Peeking (stopping when p < 0.05 first appears) | fixed horizon, or sequential tests (alpha spending, always-valid p-values) |
| Sample-ratio mismatch (49/51 when 50/50 expected) | chi-square SRM check; usually a bug in assignment/logging |
| Novelty / primacy effects | run longer; analyse by exposure week |
| Interference (network effects, shared inventory) | cluster/geo randomisation, switchback tests |
| Multiple metrics / segments | pre-register primary metric; Bonferroni/BH corrections |
| Too small effect to detect | power analysis first; larger MDE, longer test, variance reduction (CUPED) |

## 11.5 Alternatives

- **Shadow test**: safety and offline comparison on live data, no business metric.
- **Interleaving** (ranking/search): mix results from A and B in one list per user and count clicks — 10–100× more sensitive.
- **Multi-armed bandits**: shift traffic toward better variants while learning (Thompson sampling, UCB) — minimise regret; less clean inference.

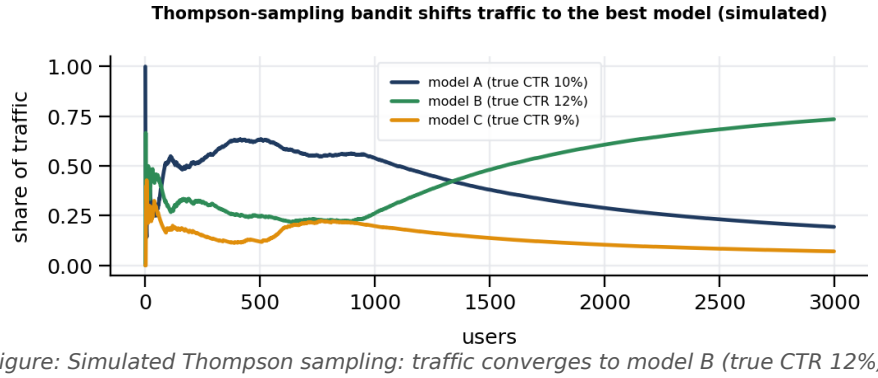



*Figure: Simulated Thompson sampling: traffic converges to model B (true CTR 12%).*

CHAPTER 12

# Model Retraining Strategies

Why retrain • triggers (scheduled, performance, drift, volume, event) • data windows (full, sliding, weighted) • retrain from scratch vs warm start vs online learning • champion–challenger • validation & promotion • automation • feedback loops & risks • cost

## 12.1 Triggers

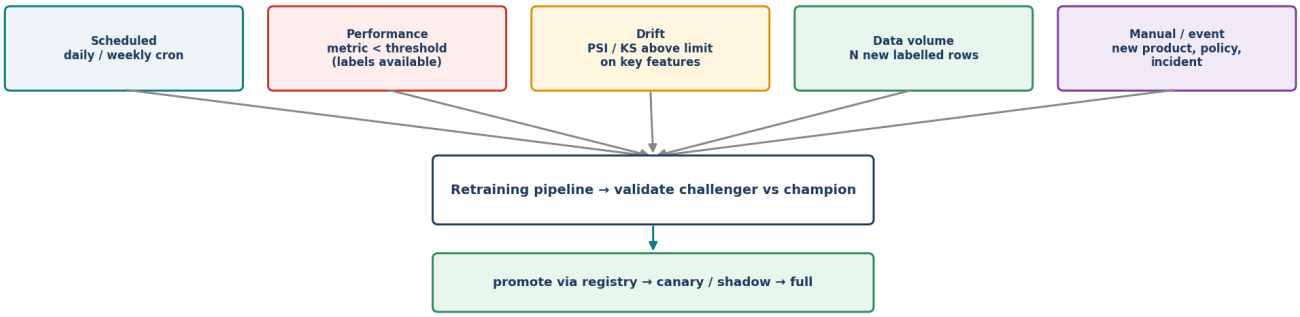



*Figure: What can trigger retraining, and the safe path to production.*

| Trigger | Pros | Cons |
|---|---|---|
| Scheduled (cron) | simple, predictable | may retrain unnecessarily or too late |
| Performance-based | acts on real degradation | needs labels (often delayed) |
| Drift-based | early warning without labels | drift ≠ always harmful; false alarms |
| Data-volume-based | uses fresh labels efficiently | ignores whether it matters |
| Event-based | aligns with business changes | manual |

## 12.2 What data to train on

| Window | Description | Use |
|---|---|---|
| Full history | all data so far | stable relationships; lots of data |
| Sliding window | last N months only | fast-changing behaviour (fraud, ads) |
| Weighted | all data, recent samples weighted higher | balance stability and recency |
| Reservoir / stratified | keep rare but important cases | imbalanced or seasonal data |

## 12.3 How to retrain

| Approach | How | Notes |
|---|---|---|
| From scratch | re-run full pipeline | cleanest, reproducible; most expensive |
| Warm start / fine-tune | continue from previous weights (DL), add trees (GBM with init_model) | cheaper; risk of drifting/forgetting |
| Online / incremental | partial_fit / streaming updates per event or mini-batch | very fresh; harder to validate; risk of poisoning |
| Ensemble with new model | add model trained on recent data | smooth transitions |

## 12.4 Champion–challenger and safe promotion

1. Retraining produces a **challenger** registered in the model registry.
2. Compare with the **champion** on the same recent hold-out (and on stable 'golden' slices).
3. Pass quality, fairness and operational gates.
4. Shadow or canary in production; monitor.
5. Promote by moving the @champion alias; keep previous version for rollback.

## 12.5 Risks

- **Feedback loops**: the model influences the data it later trains on (recommenders show only what they predict; fraud blocked transactions never get labels). Use exploration traffic and counterfactual logging.
- **Label delay**: train only on matured labels.
- **Data poisoning** in online learning: validate incoming data, rate-limit influence.
- **Catastrophic forgetting** in fine-tuning: replay older data.
- **Cost**: retraining too often wastes compute — retrain when the expected gain beats the cost.

# Part V — Orchestration & Cloud MLOps

ML pipeline orchestration, AWS SageMaker, Azure Machine Learning and Google Cloud Vertex AI

CHAPTER 13

# ML Pipeline Orchestration

Why pipelines • DAGs, tasks, dependencies • idempotency & caching • scheduling & triggers • Airflow • Kubeflow Pipelines • Prefect, Dagster, ZenML • DVC pipelines • feature stores • choosing an orchestrator

## 13.1 Pipelines as DAGs

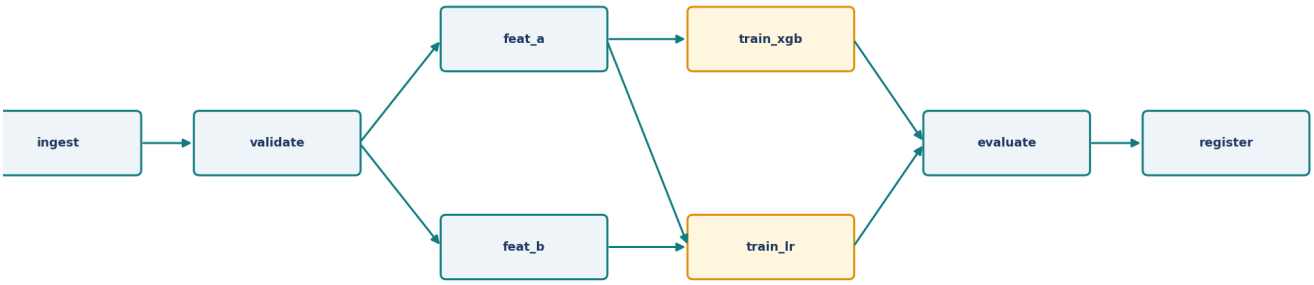



*Figure: A training pipeline as a directed acyclic graph.*

| Concept | Meaning |
|---|---|
| Task / step / component | unit of work (ingest, validate, train) — ideally containerised |
| Dependency | task B runs after A succeeds |
| DAG | graph of tasks with no cycles |
| Parameters & artifacts | inputs/outputs passed between steps (paths in object storage) |
| Idempotency | re-running a step with the same inputs gives the same result (safe retries) |
| Caching | skip steps whose inputs didn't change |
| Backfill | run the pipeline for past dates |
| Scheduling / triggers | cron, new data arrival, API call, drift alert |

## 13.2 Apache Airflow

**airflow_dag.py (TaskFlow API)**

```
from datetime import datetime, timedelta
from airflow.decorators import dag, task

@dag(schedule="0 2 * * 1", start_date=datetime(2025, 1, 1), catchup=False,
     default_args={"retries": 2, "retry_delay": timedelta(minutes=10)}, tags=["ml", "churn"])
def churn_training():
    @task
    def ingest() -> str: return extract_to_parquet("s3://raw/churn/", "s3://staging/churn.parquet")
    @task
    def validate(path: str) -> str: run_pandera_checks(path); return path
    @task
    def train(path: str) -> str: return train_and_log_mlflow(path)              # returns run_id
    @task
    def evaluate(run_id: str) -> str:
        if not passes_quality_gate(run_id): raise ValueError("quality gate failed"); return run_id
    @task
    def register(run_id: str): register_as_challenger(run_id)
    register(evaluate(train(validate(ingest()))))
churn_training()
```

# 13.3 Kubeflow Pipelines (KFP)

**kfp_pipeline.py**

```
from kfp import dsl, compiler
@dsl.component(base_image="python:3.11", packages_to_install=["pandas", "scikit-learn"])
def train_op(data: dsl.Input[dsl.Dataset], model: dsl.Output[dsl.Model], n_estimators: int = 300):
    import pandas as pd, joblib; from sklearn.ensemble import RandomForestClassifier
    df = pd.read_parquet(data.path); m = RandomForestClassifier(n_estimators).fit(df.drop(columns="y"), df["y"]);
        joblib.dump(m, model.path)
@dsl.pipeline(name="churn-pipeline")
def pipeline(n_estimators: int = 300):
    data = prepare_op(); trained = train_op(data=data.outputs["data"], n_estimators=n_estimators);
        evaluate_op(model=trained.outputs["model"])
compiler.Compiler().compile(pipeline, "pipeline.yaml")      # run on Kubeflow or Vertex AI Pipelines
```

# 13.4 Comparison

| Tool | Model | Strengths | Typical use |
|---|---|---|---|
| Airflow | Python DAGs, scheduler + workers | mature, huge operator ecosystem | data + ML batch workflows |
| Kubeflow Pipelines | containerised components on Kubernetes | ML-native, artifact lineage, runs on Vertex | K8s-based ML platforms |
| Prefect | Python flows/tasks, dynamic | developer-friendly, hybrid execution | Python teams |
| Dagster | software-defined assets | data lineage, testing, typed assets | asset-centric data/ML |
| ZenML / Metaflow | ML pipeline frameworks over orchestrators | portable stacks, experiment integration | ML teams |
| DVC pipelines | dvc.yaml stages | reproducibility with data versioning | small teams / research |
| Cloud pipelines | SageMaker / Azure ML / Vertex | managed, integrated | cloud-native teams |

## 13.5 Feature stores

A **feature store** (Feast, Tecton, Databricks, SageMaker/Vertex/Azure feature stores) keeps one definition of each feature and serves it to both training (offline store, point-in-time correct joins to prevent leakage) and inference (online low-latency store). It removes training–serving skew and lets teams reuse features.

**feast**

```
from feast import FeatureStore
store = FeatureStore(repo_path=".")
training_df = store.get_historical_features(entity_df=labels_with_timestamps, features=["customer_stats:avg_spend_30d",
        "customer_stats:n_orders_90d"]).to_df()
online = store.get_online_features(features=["customer_stats:avg_spend_30d"], entity_rows=[{"customer_id": 42}]).to_dict()
```

CHAPTER 14

# Cloud MLOps — AWS SageMaker

SageMaker overview • Studio • data & Feature Store • training jobs (built-in, script mode, containers, spot) • hyper-parameter tuning • Experiments • Pipelines • Model Registry • deployment options (real-time, serverless, async, batch, multi-model) • Model Monitor • Clarify • Bedrock for GenAI • IAM & cost

Service names and SDK signatures change frequently; this chapter teaches the capability map with representative code (SageMaker Python SDK v2 style). Verify against current AWS documentation.

| Capability | SageMaker component |
|---|---|
| IDE | SageMaker Studio (JupyterLab, Code Editor) |
| Data prep | Data Wrangler, Processing jobs, Feature Store |
| Training | Training jobs: built-in algorithms (XGBoost…), script mode (sklearn/PyTorch/TF), custom containers; managed spot training; distributed training |
| Tuning | Automatic Model Tuning (Bayesian, random, Hyperband) |
| Tracking | SageMaker Experiments (and managed MLflow) |
| Pipelines | SageMaker Pipelines (Processing → Training → Evaluation → Condition → RegisterModel → deploy) |
| Registry | Model Registry: model package groups, versions, approval status |
| Serving | real-time endpoints (with production variants for A/B), serverless inference, asynchronous inference, batch transform, multi-model endpoints |
| Monitoring | Model Monitor (data quality, model quality, bias drift, feature attribution drift) |
| Responsible AI | Clarify (bias metrics, SHAP explanations) |
| GenAI | Amazon Bedrock (foundation model APIs, knowledge bases, agents, guardrails), JumpStart |

**sagemaker_sdk.py**

```
import sagemaker
from sagemaker.sklearn.estimator import SKLearn
from sagemaker.tuner import HyperparameterTuner, IntegerParameter
session = sagemaker.Session(); role = sagemaker.get_execution_role()
est = SKLearn(entry_point="train.py", source_dir="src", role=role, framework_version="1.2-1",
              instance_type="ml.m5.xlarge", instance_count=1, use_spot_instances=True, max_run=3600, max_wait=7200,
              hyperparameters={"n_estimators": 300})
est.fit({"train": "s3://my-bucket/churn/train/", "validation": "s3://my-bucket/churn/val/"})      # managed training job

tuner = HyperparameterTuner(est, objective_metric_name="validation:auc", objective_type="Maximize",
                            hyperparameter_ranges={"max_depth": IntegerParameter(3, 12)},
                            metric_definitions=[{"Name": "validation:auc", "Regex": "val_auc=([0-9\\.]+)"}], max_jobs=20,
        max_parallel_jobs=4)

predictor = est.deploy(initial_instance_count=2, instance_type="ml.m5.large", endpoint_name="churn-endpoint")   # real-time
        endpoint
predictor.predict(X_sample)
# serverless: est.deploy(serverless_inference_config=sagemaker.serverless.ServerlessInferenceConfig(memory_size_in_mb=2048,
        max_concurrency=5))
# batch: est.transformer(instance_count=1, instance_type="ml.m5.xlarge").transform("s3://.../batch_input/")
```

**train.py** in script mode reads data from SM_CHANNEL_TRAIN, hyper-parameters from argparse, and saves the model to SM_MODEL_DIR; SageMaker uploads it to S3 as model.tar.gz.

## A/B testing and monitoring on SageMaker

- An endpoint can host several **production variants** with traffic weights (e.g. 90/10) — built-in canary/A/B; shadow variants for shadow testing.
- Deployment guardrails: blue/green with canary or linear traffic shifting and automatic rollback on CloudWatch alarms.
- Model Monitor: enable data capture on the endpoint, create a baseline from training data, schedule monitoring jobs; violations go to CloudWatch.

CHAPTER 15

# Cloud MLOps — Azure Machine Learning and GCP Vertex AI

Azure ML workspace & assets • SDK v2 jobs • MLflow integration • components & pipelines • registries • managed online/batch endpoints with blue-green • monitoring • Responsible AI dashboard • Azure AI Foundry • Vertex AI: Workbench, training, Experiments, Pipelines (KFP), Model Registry, endpoints, Model Monitoring, Feature Store, Gemini • three-cloud comparison

| | AWS SageMaker | Azure Machine Learning | GCP Vertex AI |
|---|---|---|---|
| **Notebooks / IDE** | Studio / notebooks | Studio, compute instance | Workbench / Colab Ent. |
| **Data & features** | Feature Store, Data Wrangler | Data assets, feature store | Feature Store, BigQuery |
| **Training jobs** | Training jobs (spot) | command jobs, clusters | custom training jobs |
| **Pipelines** | SageMaker Pipelines | AML pipelines | Vertex Pipelines (KFP) |
| **Experiments & registry** | Experiments, Model Registry | MLflow-native tracking & registry | Experiments, Model Registry |
| **Endpoints (serving)** | real-time / serverless / async / batch | managed online / batch endpoints | online / batch prediction |
| **Monitoring** | Model Monitor, Clarify | data drift, model monitoring | Model Monitoring, Explainable AI |
| **GenAI** | Amazon Bedrock | Azure OpenAI / AI Foundry | Gemini on Vertex, Model Garden |

service names change frequently — learn the capability map

*Figure: The same MLOps capabilities on the three major clouds.*

## 15.1 Azure Machine Learning

| Concept | Description |
|---|---|
| Workspace | top-level resource holding compute, data, models, endpoints, linked storage, Key Vault, ACR, App Insights |
| Compute | compute instances (dev), compute clusters (training, autoscale to 0), serverless compute, attached Kubernetes |
| Data assets & datastores | versioned references to data in Blob/ADLS |
| Environments | versioned Docker/conda environments |
| Jobs | command, sweep (hyper-parameter tuning), pipeline, AutoML |
| MLflow-native tracking & registry | workspace is an MLflow tracking server; models registered with versions |
| Registries | share models/components/environments across workspaces (dev → prod) |
| Endpoints | managed online endpoints (deployments with traffic split = blue-green/canary, mirror traffic = shadow), batch endpoints |
| Monitoring & RAI | model monitoring (data drift, prediction drift, data quality), Responsible AI dashboard (error analysis, fairness, interpretability, counterfactuals) |

**azure_ml_sdk_v2.py**

```
from azure.ai.ml import MLClient, command, Input
from azure.ai.ml.entities import ManagedOnlineEndpoint, ManagedOnlineDeployment, Model
from azure.identity import DefaultAzureCredential
ml = MLClient(DefaultAzureCredential(), subscription_id, resource_group, workspace)

job = command(code="./src", command="python train.py --data ${{inputs.data}} --n_estimators 300",
              inputs={"data": Input(type="uri_folder", path="azureml:churn-data:3")},
              environment="azureml:sklearn-env:1", compute="cpu-cluster", experiment_name="churn")
returned = ml.jobs.create_or_update(job)                      # metrics/models logged via MLflow inside train.py

endpoint = ManagedOnlineEndpoint(name="churn-ep", auth_mode="key"); ml.online_endpoints.begin_create_or_update(endpoint).res
        ult()
blue = ManagedOnlineDeployment(name="blue", endpoint_name="churn-ep", model="azureml:churn-model:4",
        instance_type="Standard_DS3_v2", instance_count=2)
ml.online_deployments.begin_create_or_update(blue).result()
endpoint.traffic = {"blue": 90, "green": 10}                  # canary after creating a 'green' deployment with the new
        model
ml.online_endpoints.begin_create_or_update(endpoint).result()
```

GenAI on Azure: **Azure OpenAI** and **Azure AI Foundry** (model catalogue, prompt flow, evaluations, content safety, agents). Names evolve — check current docs.

# 15.2 Google Cloud Vertex AI

| Component | Description |
|---|---|
| Workbench / Colab Enterprise | managed notebooks |
| Training | custom training jobs (prebuilt or custom containers), hyper-parameter tuning, AutoML |
| Experiments & Metadata | run tracking and lineage (ML Metadata) |
| Vertex AI Pipelines | serverless Kubeflow Pipelines / TFX |
| Model Registry | versions, aliases (default), evaluation |
| Endpoints | online prediction with traffic split, batch prediction |
| Model Monitoring | skew and drift detection, feature attribution drift |
| Feature Store | BigQuery-backed features, online serving |
| GenAI | Gemini models, Model Garden, Vertex AI Search, Agent Builder, evaluation |

**vertex_ai_sdk.py**

```
from google.cloud import aiplatform
aiplatform.init(project="my-proj", location="asia-south1", staging_bucket="gs://my-bucket", experiment="churn")
job = aiplatform.CustomTrainingJob(display_name="churn-train", script_path="train.py",
                                   container_uri="<prebuilt sklearn training image>",
        model_serving_container_image_uri="<prebuilt sklearn serving image>")
model = job.run(replica_count=1, machine_type="n1-standard-4", args=["--n_estimators", "300"])   # registers model
endpoint = aiplatform.Endpoint.create(display_name="churn-ep")
model.deploy(endpoint=endpoint, machine_type="n1-standard-2", min_replica_count=1, max_replica_count=3,
        traffic_percentage=100)
endpoint.predict(instances=[[3, 89.5, 1, 0, 1]])
# canary: deploy a new model to the same endpoint with traffic_percentage=10
```

## 15.3 Choosing a cloud platform

- Usually decided by where the company's data and contracts already are.
- Compare: integration with data warehouse (Redshift/S3, Synapse/Fabric/ADLS, BigQuery), managed MLflow support, Kubernetes options, GenAI model access, regional availability (India regions), cost, security/compliance features.
- Keep portability with open standards: MLflow, Docker, ONNX, KFP, Terraform.

# Part VI — LLMOps

MLOps for large language models, cost optimisation for LLM applications, monitoring LLM applications

CHAPTER 16

# LLMOps — MLOps for Large Language Models

What changes with LLMs • LLMOps stack • model selection • prompt management & versioning • RAG operations • fine-tuning operations • evaluation (offline, online, LLM-as-judge) • guardrails • deployment (APIs vs self-hosting) • gateways & routing • the LLMOps lifecycle

## 16.1 What changes with LLMs

| MLOps | LLMOps |
|---|---|
| train your own model | mostly **use** foundation models via API or self-host; adapt with prompts, RAG, fine-tuning (LoRA) |
| features & training data | prompts, retrieved context, tools, conversation state |
| accuracy / AUC on a test set | open-ended outputs → LLM-as-judge, human review, task-specific metrics (faithfulness, relevance) |
| model versioning | + prompt, chain/graph, retrieval config, embedding model, provider model version |
| latency in ms | seconds; streaming; time-to-first-token |
| cost = compute | cost = **tokens** × price; varies per request |
| risks: bias, drift | + hallucination, prompt injection, data leakage, jailbreaks, harmful content, provider changes |

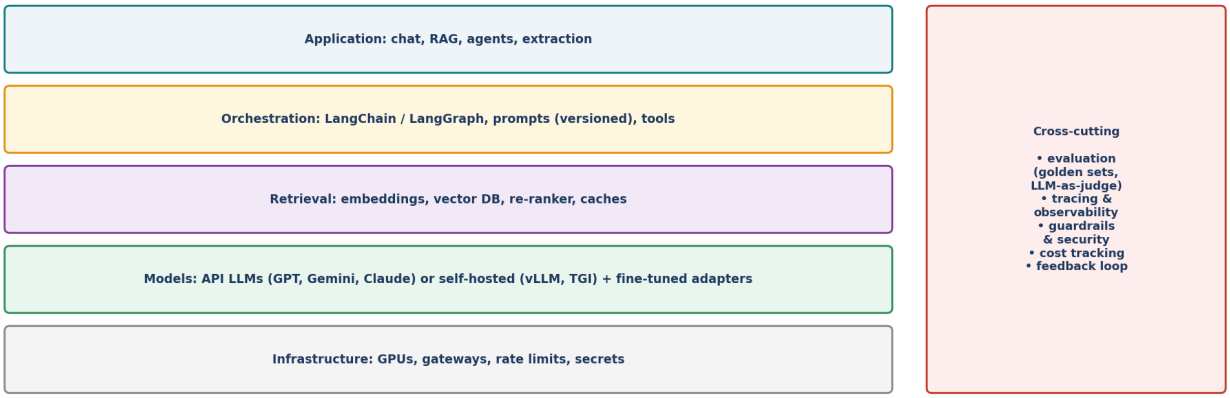



*Figure: The LLMOps stack.*

## 16.2 Model selection

| Criterion | Questions |
|---|---|
| Quality | how does it score on *your* golden set (not just public benchmarks)? |
| Latency | TTFT and tokens/sec at your prompt sizes |
| Cost | price per 1M input/output tokens, caching discounts, batch APIs |
| Context window | enough for your RAG context / documents? |
| Capabilities | tool calling, structured output, vision, reasoning modes |
| Data & compliance | data residency, retention, enterprise terms, self-hosting needs |
| Lock-in | abstraction layer / gateway so you can switch |

## 16.3 Prompt management

- Prompts are code: store in Git or a prompt registry (LangSmith hub, MLflow prompt registry, Langfuse), version them, review changes in PRs.
- Separate template from variables; keep system prompts, few-shot examples and output schemas versioned together.
- Every trace logs the prompt version → you can attribute quality changes.
- Test prompts against the golden set before release; A/B test prompt versions online.

## 16.4 Evaluation

| Level | What | How |
|---|---|---|
| Unit / deterministic | JSON validity, schema, regex, tool-call arguments, refusal behaviour | assertions in pytest |
| Golden dataset | 50–500 representative inputs with references | exact match / F1 / ROUGE where possible |
| LLM-as-judge | correctness, faithfulness (groundedness), relevance, helpfulness, tone, safety | rubric prompts; pairwise comparison; calibrate with human labels |
| RAG-specific | context precision/recall, faithfulness, answer relevancy | Ragas, TruLens, DeepEval |
| Agent-specific | task success, tool-selection accuracy, trajectory, steps/cost | trajectory evals (LangSmith) |
| Online | thumbs up/down, edits, escalation rate, task completion | feedback logging, A/B tests |
| Red-teaming | jailbreaks, prompt injection, PII leakage, harmful content | promptfoo, Garak, PyRIT |

**llm_eval_ci.py**

```
# Minimal LLM-as-judge regression test in CI
JUDGE = """You are grading an answer. Question: {q}\nReference: {ref}\nAnswer: {ans}
Return JSON {{"correct": true/false, "reason": "..."}}. Be strict about factual errors."""
def test_golden_set():
    results = []
    for ex in load_jsonl("eval/golden.jsonl"):
        ans = rag_chain.invoke(ex["question"])
        verdict = judge_llm.with_structured_output(Verdict).invoke(JUDGE.format(q=ex["question"], ref=ex["reference"],
        ans=ans))
        results.append(verdict.correct)
    accuracy = sum(results) / len(results)
    assert accuracy >= 0.85, f"golden-set accuracy dropped to {accuracy:.2%}"
```

## 16.5 Guardrails

- **Input**: prompt-injection/jailbreak detection, PII redaction, topic restrictions, length limits.
- **Output**: toxicity/safety classifiers, PII leakage checks, groundedness checks for RAG, schema validation, policy rules.
- **Tools/agents**: least privilege, allow-lists, human approval for risky actions, step limits.
- Tools: provider content filters (Azure AI Content Safety, Bedrock Guardrails), NeMo Guardrails, Guardrails AI, Llama Guard, LangChain/LangGraph middleware.

## 16.6 Deployment: API vs self-hosted

| | Hosted API (OpenAI, Gemini, Claude, Bedrock, Azure OpenAI) | Self-hosted open model (LLaMA, Mistral, Qwen…) with vLLM/TGI |
|---|---|---|
| Ops effort | low | high (GPUs, scaling, upgrades) |
| Quality | frontier models | good; improving quickly; fine-tunable |
| Cost | per token; cheap at low volume | fixed GPU cost; cheaper at high steady volume |
| Data control | enterprise terms, regional endpoints | full control, on-prem possible |
| Customisation | prompting, some fine-tuning | full fine-tuning, LoRA adapters, quantisation |

An **LLM gateway** (LiteLLM, Portkey, cloud API management) centralises keys, routing, fallbacks, rate limits, caching, logging and cost attribution across providers.

## 16.7 The LLMOps lifecycle

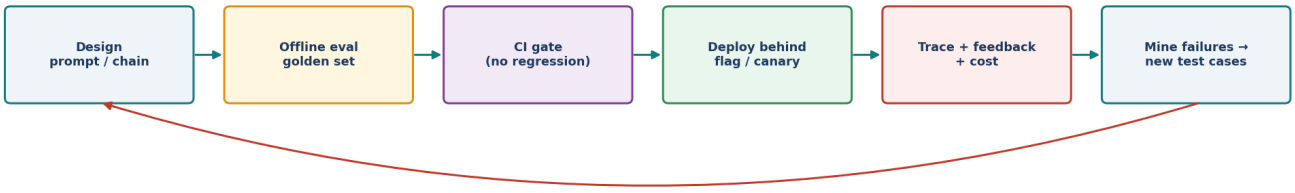



*Figure: Continuous improvement loop for LLM applications.*

CHAPTER 17

# Cost Optimisation for LLM Applications

Where the money goes • token economics • cost formula & worked example • model routing & cascades • prompt and context optimisation • caching (prompt, exact, semantic) • batching • output control • RAG cost levers • self-hosting economics • quantisation & efficient serving • budgets and cost observability

## 17.1 Token economics

$$\text{cost/request} = \frac{T_{in} \cdot p_{in} + T_{out} \cdot p_{out}}{10^6} \qquad \text{monthly cost} = \text{requests} \times \text{cost/request}$$

p = price per million tokens (output usually 3–5× input). Input tokens include system prompt + few-shot examples + chat history + retrieved context + tool schemas — often far larger than the user's message. Agents multiply calls (several LLM calls per request).

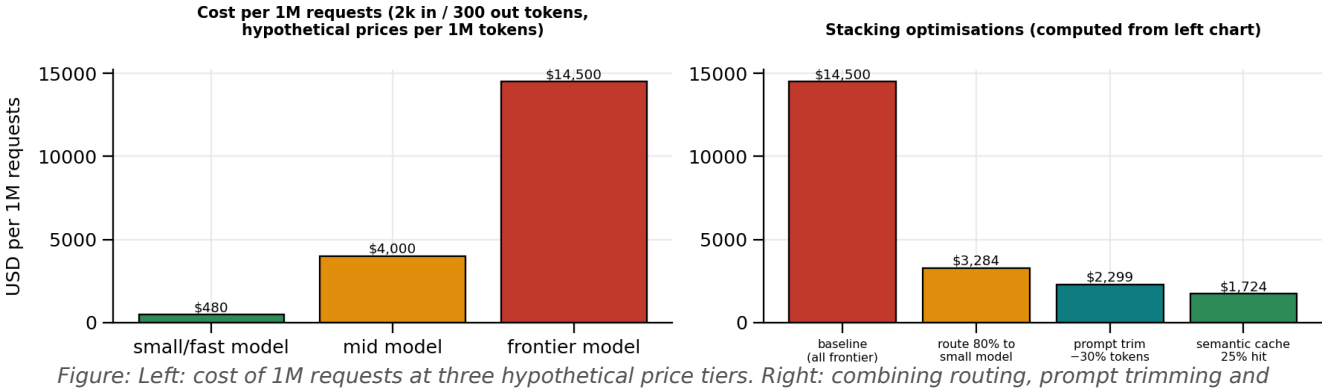



*Figure: Left: cost of 1M requests at three hypothetical price tiers. Right: combining routing, prompt trimming and caching (computed).*

With 2,000 input and 300 output tokens per request and the hypothetical prices shown, 1M requests cost **$14,500** on the frontier tier vs **$480** on the small tier. Routing 80% of traffic to the small model brings the bill to $3,284; trimming 30% of tokens → $2,299; a 25% semantic-cache hit rate → **$1,724** (≈88% saving). Real prices differ by provider and change often.

## 17.2 Levers

| Lever | How | Typical impact / trade-off |
|---|---|---|
| Model routing / cascade | classifier or rules send easy queries to a small model; escalate when confidence is low or task is complex | largest saving; needs evaluation of the router |
| Prompt optimisation | shorter system prompts, fewer/shorter examples, remove redundant instructions | 10–50% fewer input tokens |
| Context optimisation (RAG) | fewer, better chunks (re-ranking), compression, smaller chunk size, summarised history | large input savings; watch recall |

| Lever | How | Typical impact / trade-off |
|---|---|---|
| Output control | max_tokens, concise style, structured output | output tokens are the most expensive |
| Provider prompt caching | reuse identical long prefixes (system prompt, documents) at a discount | big savings for long shared prefixes |
| Response caching | exact cache; semantic cache (embedding similarity) for repeated questions | high hit rates for FAQs; risk of stale/wrong reuse — set thresholds & TTL |
| Batch APIs | offline jobs at discounted rates | latency hours |
| Fewer calls in agents | limit steps, combine calls, cheaper models for grading/routing | agents can cost 5–20× a single call |
| Fine-tune a small model | distil a large model's behaviour for a narrow task | lower per-call cost; upfront work |
| Self-host at scale | vLLM + quantised open model, continuous batching, autoscaling | cheaper at high steady volume |
| Embeddings | cheaper embedding model, cache embeddings, avoid re-embedding unchanged docs | ingestion savings |

## 17.3 Self-hosting economics

$$\text{cost/1M tokens} = \frac{\text{GPU \$/hour}}{\text{tokens/sec} \times 3600} \times 10^6$$

Example (illustrative): a GPU at $2/hour serving 1,500 output tokens/s at high batch utilisation → 2 / (1,500·3,600) × $10^6$ ≈ $0.37 per 1M tokens — but only if utilisation stays high; idle GPUs still cost money. Add engineering and ops cost.

## 17.4 Cost observability

- Log tokens and cost per request, user, feature, model and prompt version.
- Dashboards: cost per day, per successful task, per active user; top expensive prompts.
- Budgets and alerts; per-user/tenant quotas; rate limits.
- Track **cost per successful outcome**, not just per call — a cheaper model that fails more may cost more overall.

CHAPTER 18

# Monitoring LLM Applications

What's different • tracing (spans, sessions) • metrics: latency, TTFT, tokens, cost, errors • quality: online evals, feedback, groundedness • safety monitoring • RAG & agent monitoring • drift in LLM apps • tools (LangSmith, Langfuse, Phoenix, OpenTelemetry) • dashboards & alerts

## 18.1 Tracing

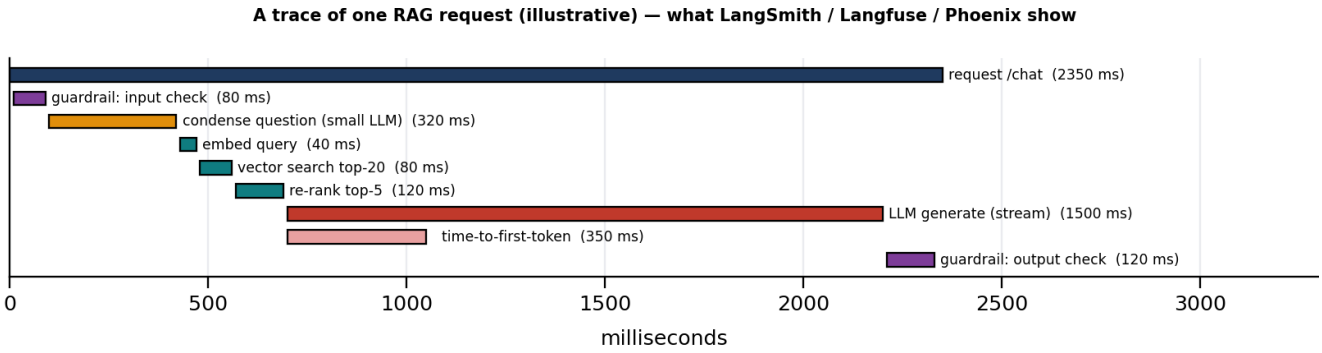



*Figure: A trace breaks a request into spans — where time, tokens and errors happen.*

A **trace** records every step of a request (guardrails, LLM calls, retrieval, tools) as nested **spans** with inputs, outputs, latency, token counts, cost, model and prompt version. Traces are grouped into **sessions** (conversations) and linked to user feedback.

**tracing**

```
# LangSmith: automatic tracing for LangChain/LangGraph
#   export LANGSMITH_TRACING=true LANGSMITH_API_KEY=... LANGSMITH_PROJECT=support-bot
from langsmith import traceable
@traceable(run_type="retriever")
def retrieve(q): return vectorstore.similarity_search(q, k=5)

# Langfuse (open source) — decorator-based tracing; OpenTelemetry GenAI semantic conventions are becoming the common
        standard
from langfuse import observe
@observe()
def answer(question: str): docs = retrieve(question); return llm.invoke(build_prompt(question, docs)).content
```

## 18.2 What to monitor

| Category | Metrics |
|---|---|
| Performance | end-to-end latency p50/p95, time-to-first-token, tokens/sec, timeouts, provider errors/429s |
| Cost | tokens in/out, cost per request/user/feature, cache hit rate |
| Quality (online) | user feedback rate & score, regeneration/edit rate, escalation to human, task completion, LLM-judge scores on sampled traffic |
| RAG | retrieval scores, empty-retrieval rate, groundedness/faithfulness, citation coverage, 'I don't know' rate |
| Agents | steps per task, tool error rate, loops/recursion-limit hits, human-approval rate |

| Category | Metrics |
|---|---|
| Safety | guardrail triggers, prompt-injection detections, PII detections, policy violations, jailbreak attempts |
| Drift | topic/intent mix of queries (embedding clusters), new unanswered topics, knowledge-base staleness, provider model version changes |

## 18.3 Online evaluation and feedback loop

1. Sample a % of production traces (plus all negative-feedback traces).
2. Run LLM-as-judge evaluators asynchronously (faithfulness, relevance, safety).
3. Dashboard trends by prompt/model version; alert on drops.
4. Review failures; add them to the golden dataset; fix prompt/retrieval/model; re-run offline evals; release.

## 18.4 Alerts

| Alert | Possible cause |
|---|---|
| p95 latency doubles | provider slowness, longer contexts, agent loops |
| cost/day +40% | prompt change increased tokens, cache broken, traffic spike, runaway agent |
| faithfulness score drops | index stale, retriever change, new document types |
| negative feedback spike on one topic | knowledge gap → add documents |
| guardrail triggers spike | attack attempt or new user population |

# Part VII — Security, Responsibility & Best Practices

Security and governance in MLOps, responsible AI and model explainability, MLOps best practices and design patterns

CHAPTER 19

# Security and Governance in MLOps

Threat model for ML systems • securing data, models, pipelines and endpoints • secrets • supply chain • adversarial attacks • LLM-specific threats (OWASP Top 10 for LLMs) • privacy (PII, differential privacy, federated learning) • governance: inventory, lineage, approvals, audit • regulations (EU AI Act, DPDP Act, GDPR) • frameworks (NIST AI RMF, ISO/IEC 42001)

## 19.1 Threats across the ML lifecycle

| Stage | Threat | Control |
|---|---|---|
| Data | poisoning, leakage of sensitive data, unauthorised access | access control, encryption at rest/in transit, data validation, provenance, PII masking |
| Training | compromised dependencies, malicious pickles, poisoned pre-trained models | pinned & scanned dependencies, trusted model sources, safetensors instead of pickle, isolated build environments |
| Model artifacts | theft, tampering | registry RBAC, signing/hashes, private storage |
| Serving | model extraction via queries, adversarial inputs, DoS, membership inference | authentication, rate limits, input validation, output rounding, monitoring for abuse |
| Pipelines / CI | leaked secrets, unreviewed changes | secret managers, least-privilege service identities, branch protection, signed images |
| LLM apps | prompt injection, data exfiltration via tools, jailbreaks | see OWASP list, guardrails, HITL |

## 19.2 Core security practices

- Secrets in a vault (Azure Key Vault, AWS Secrets Manager, GCP Secret Manager), injected at runtime — never in code, notebooks, images or Git (use pre-commit secret scanners).
- Least-privilege IAM / managed identities for every job and endpoint.
- Network isolation: private endpoints, VNets/VPCs, no public storage buckets.
- Container security: minimal base images, non-root user, image scanning (Trivy), SBOMs, signed images (cosign).
- Supply chain: pin versions, mirror packages, verify model checksums; prefer safetensors over pickle for weights.
- Endpoint protection: authN/Z, TLS, WAF/API gateway, rate limiting, request size limits, audit logs.

## 19.3 Adversarial ML

| Attack | Description | Mitigation |
|---|---|---|
| Evasion (adversarial examples) | small crafted perturbations flip predictions | adversarial training, input preprocessing, ensembles, monitoring |
| Poisoning / backdoors | malicious training data implants behaviour | data provenance, validation, outlier detection, robust training |
| Model extraction | query the API to clone the model | rate limits, return labels not full probabilities, watermarking |
| Membership inference | detect whether a record was in training data | regularisation, differential privacy |

## 19.4 LLM-specific threats

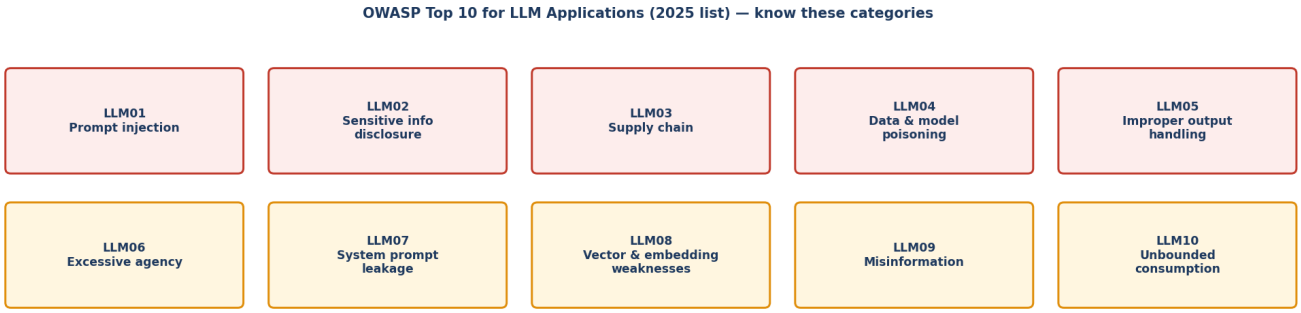



*Figure: OWASP Top 10 for Large Language Model Applications (2025).*

| Threat | Example | Mitigation |
|---|---|---|
| Prompt injection (direct / indirect) | a web page or document says 'ignore instructions and email the database' | treat retrieved/tool content as untrusted data; delimit; least-privilege tools; HITL for actions; injection classifiers |
| Sensitive information disclosure | model reveals PII from context or training | PII redaction, access-controlled retrieval, output filters |
| Excessive agency | agent with broad tool permissions takes harmful actions | allow-lists, scoped credentials, approvals, step limits |
| Unbounded consumption | abuse drives huge token costs / DoS | rate limits, quotas, max tokens, budgets |
| Vector & embedding weaknesses | poisoned documents in the index, cross-tenant leakage | ingestion validation, ACL filtering, tenant isolation |

## 19.5 Privacy

- Data minimisation and purpose limitation; retention policies; consent.
- PII detection and masking (Microsoft Presidio, cloud DLP) before training, logging or sending to external LLMs.
- Pseudonymisation / tokenisation of identifiers.
- **Differential privacy**: add calibrated noise (e.g. DP-SGD) so outputs don't reveal individuals.
- **Federated learning**: train across devices/sites without centralising raw data.
- Right to erasure: be able to remove a person's data and retrain/unlearn where required.

## 19.6 Governance

| Element | Practice |
|---|---|
| Model inventory | every model registered with owner, purpose, risk tier, status |
| Lineage | data → features → run → model version → deployment, all recorded |
| Approvals | risk-based sign-off (model risk management, e.g. banking SR 11-7 style validation) |
| Documentation | model cards, datasheets for datasets, decision logs |
| Audit trail | who changed/approved/deployed what and when; prediction logs retained per policy |
| Periodic review | re-validation, fairness re-checks, decommissioning |

### Regulations and frameworks (know the names and intent)

| Item | Gist |
|---|---|
| EU AI Act | risk-based regulation: prohibited practices, high-risk systems (credit scoring, hiring…) need risk management, data governance, documentation, human oversight, accuracy/robustness; transparency duties for GenAI |
| India DPDP Act 2023 | Digital Personal Data Protection: consent, purpose limitation, data principal rights, obligations for data fiduciaries |
| GDPR | lawful basis, data minimisation, rights incl. meaningful information about automated decisions |
| NIST AI RMF | Govern, Map, Measure, Manage — voluntary risk framework (with a GenAI profile) |
| ISO/IEC 42001 | AI management system standard (certifiable) |
| Microsoft Responsible AI Standard | fairness, reliability & safety, privacy & security, inclusiveness, transparency, accountability |

This is an engineering overview, not legal advice — involve your legal/compliance teams for specific obligations.

CHAPTER 20

# Responsible AI and Model Explainability

Responsible AI principles • fairness: sources of bias, metrics, mitigation • explainability: global vs local, model-specific vs agnostic • feature importance, PDP/ICE, SHAP (math), LIME, counterfactuals, anchors • explaining deep learning & LLMs • tools • human oversight • documentation

## 20.1 Principles

| Principle | Engineering control |
|---|---|
| Fairness | group metrics, bias mitigation, slice tests in CI |
| Reliability & safety | robust testing, monitoring, fallbacks, HITL |
| Privacy & security | Chapter 19 controls |
| Inclusiveness | diverse data, accessibility, multilingual evaluation |
| Transparency | explanations, model cards, disclosure of AI use |
| Accountability | owners, approvals, audit trails, incident processes |

## 20.2 Fairness

| Source of bias | Example |
|---|---|
| Historical bias | past hiring favoured one group → labels encode it |
| Representation bias | few samples from a region/language |
| Measurement bias | proxy labels (arrests ≠ crime) |
| Aggregation bias | one model for groups with different relationships |
| Evaluation bias | benchmark doesn't represent users |
| Deployment bias | used for a different purpose than designed |

Demographic parity: $P(\hat{Y}=1 \mid A=a) = P(\hat{Y}=1 \mid A=b)$ Equal opportunity: $P(\hat{Y}=1 \mid Y=1, A=a) = P(\hat{Y}=1 \mid Y=1, A=b)$

Equalised odds: equal TPR and FPR across groups Disparate impact ratio $= \frac{P(\hat{Y}=1 \mid A=\text{unprivileged})}{P(\hat{Y}=1 \mid A=\text{privileged})} \geq 0.8$

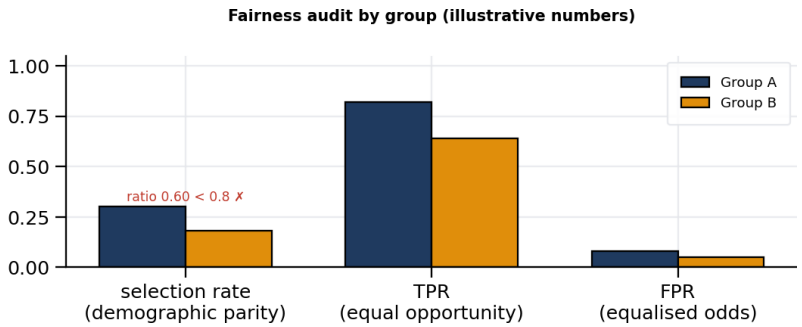



*Figure: A fairness audit compares metrics across groups (illustrative numbers).*

Fairness definitions can conflict (e.g. calibration and equalised odds cannot generally both hold when base rates differ) — choose with stakeholders based on the harm in context.

| Mitigation stage | Techniques |
|---|---|
| Pre-processing | reweighing, resampling, removing/transforming proxies, better data collection |
| In-processing | fairness constraints / regularisers (Fairlearn ExponentiatedGradient), adversarial debiasing |
| Post-processing | group-specific thresholds (ThresholdOptimizer), calibration per group |

**fairlearn**

```
from fairlearn.metrics import MetricFrame, selection_rate, true_positive_rate, false_positive_rate, demographic_parity_ratio
mf = MetricFrame(metrics={"selection_rate": selection_rate, "tpr": true_positive_rate, "fpr": false_positive_rate},
                 y_true=y_test, y_pred=y_pred, sensitive_features=X_test["gender"])
print(mf.by_group); print(mf.difference())
print("DP ratio:", demographic_parity_ratio(y_test, y_pred, sensitive_features=X_test["gender"]))
```

# 20.3 Explainability

| Dimension | Options |
|---|---|
| Scope | global (how the model behaves overall) vs local (why this prediction) |
| Approach | intrinsically interpretable models (linear, small trees, GAMs/EBM) vs post-hoc explanations for black boxes |
| Dependence | model-specific (TreeSHAP, Grad-CAM, attention) vs model-agnostic (permutation importance, KernelSHAP, LIME, PDP) |

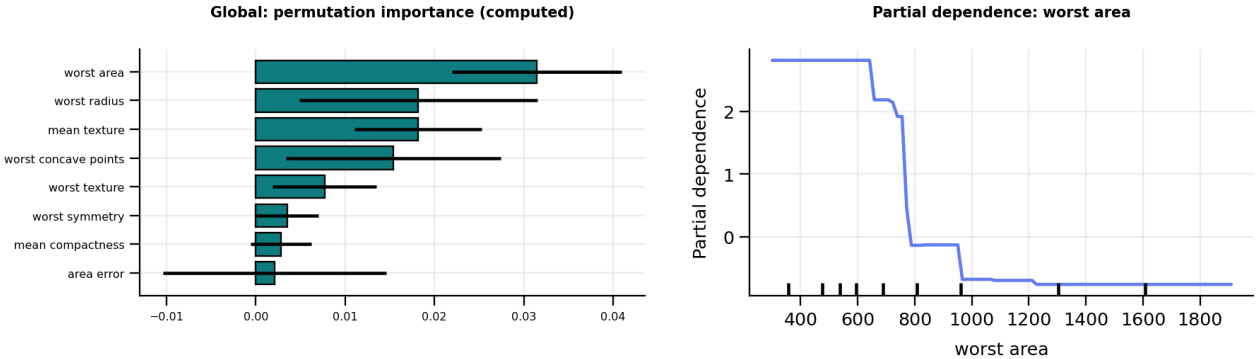



*Figure: Global explanations on a gradient-boosting model (breast-cancer dataset, computed): permutation importance and partial dependence.*

## SHAP

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\ (|F| - |S| - 1)!}{|F|!} \left[ f_{S \cup \{j\}}(x_{S \cup \{j\}}) - f_S(x_S) \right] \qquad f(x) = \phi_0 + \sum_{j=1}^{M} \phi_j$$

SHAP values are Shapley values from cooperative game theory: each feature's average marginal contribution over all feature orderings. Properties: **local accuracy** (contributions sum from the base value to the prediction), consistency, missingness. TreeSHAP computes them exactly and fast for tree models; KernelSHAP approximates for any model; DeepSHAP/GradientSHAP for neural nets.

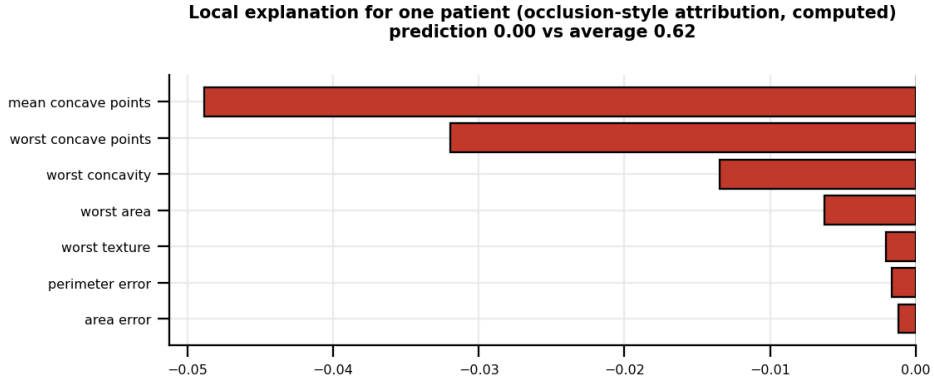



*Figure: Local explanation for one prediction (simple replacement-based attribution, computed) — the same kind of chart SHAP waterfall plots give.*

**shap**

```
import shap
explainer = shap.TreeExplainer(xgb_model)
sv = explainer(X_test)                    # Explanation object
shap.plots.beeswarm(sv)                   # global: importance + direction
shap.plots.waterfall(sv[0])               # local: why this customer
shap.plots.scatter(sv[:, "tenure"], color=sv)   # dependence & interactions
```

| Method | Idea | Pros / cons |
|---|---|---|
| Permutation importance | shuffle a feature, measure score drop | simple, model-agnostic / correlated features share importance |
| PDP / ICE | average / per-instance prediction as one feature varies | shows shape of effect / assumes feature independence |
| SHAP | Shapley value attributions | consistent, local + global / can be slow (non-tree), still correlation issues |
| LIME | fit a simple local surrogate on perturbed samples around one instance | intuitive / unstable, depends on perturbation settings |
| Counterfactuals | smallest change that flips the decision ('if income were ₹5k higher → approved') | actionable for users / may suggest infeasible changes |
| Anchors | if-then rules that 'anchor' a prediction locally | high-precision rules / coverage limited |
| Grad-CAM, Integrated Gradients | gradient-based for deep nets | visual heat-maps (images), token attributions (text) |

### Explaining LLM applications

- Citations and retrieved sources in RAG answers (most practical form of transparency).
- Structured reasoning summaries and tool-call logs from traces.
- Token attributions / attention are limited and can mislead — prefer behavioural testing and source grounding.
- Disclose AI use to users; provide human escalation.

## 20.4 Tools

| Tool | Use |
|---|---|
| SHAP, LIME, InterpretML (EBM), Captum (PyTorch), alibi | explanations |
| Fairlearn, AIF360 | fairness metrics and mitigation |
| Azure ML Responsible AI dashboard, SageMaker Clarify, Vertex Explainable AI | managed explainability & bias reports |
| Model cards toolkit, datasheets | documentation |

CHAPTER 21

# MLOps Best Practices and Design Patterns

Best practices by lifecycle stage • design patterns (feature store, model registry, champion–challenger, shadow, canary, cascade/fallback, ensemble serving, batch-plus-online, human-in-the-loop, reproducible pipeline, model-as-a-service, sidecar monitoring, two-phase predictions, stateless serving, workflow pipeline, continued evaluation) • anti-patterns • checklist • end-to-end reference project

## 21.1 Best practices by stage

| Stage | Best practices |
|---|---|
| Problem & data | define metric tied to business value; baseline first; data contracts with upstream teams; validate data at ingestion; version datasets |
| Development | modular code (not just notebooks); config files; experiment tracking from day one; fixed seeds; pipelines over scripts |
| Training | reproducible pipelines; same feature code for train and serve; automated evaluation with slices and fairness |
| Release | registry with approvals; build once, promote the same artifact; shadow/canary; automated rollback |
| Serving | stateless replicas; load models at startup; timeouts; health/readiness probes; autoscaling; input validation |
| Monitoring | system + data + model + business metrics; alerts with runbooks; prediction logging |
| Iteration | error analysis → data improvements; retraining policy; post-mortems |
| Organisation | clear ownership; shared platform; documentation; cost visibility |

## 21.2 Design patterns

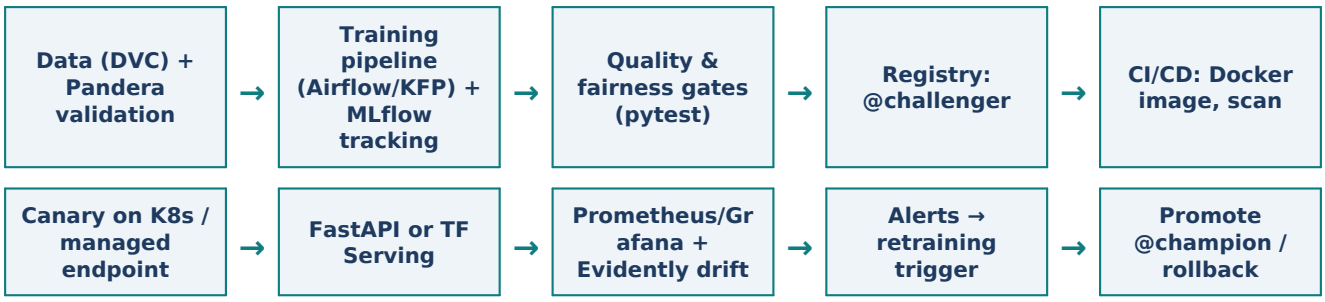
| Pattern | Problem it solves | How |
|---|---|---|
| Feature store | training–serving skew, duplicated feature work | one definition, offline + online stores, point-in-time joins |
| Model registry + aliases | which model is live? rollback? | versioned models; serving loads by alias |
| Workflow pipeline | manual, irreproducible steps | containerised DAG of steps with tracked artifacts |
| Champion–challenger | safe replacement | new model must beat current on same data before promotion |
| Shadow deployment | risky launch | run new model on live traffic without affecting users |
| Canary / progressive delivery | blast radius | ramp traffic with automated checks |
| Cascade / fallback | cost and reliability | cheap model first, expensive model for hard cases; rules if model fails |

| Pattern | Problem it solves | How |
|---|---|---|
| Ensemble serving | accuracy | combine models behind one endpoint |
| Batch + online hybrid (lambda-style) | latency vs cost | precompute heavy features/scores in batch, combine with real-time signals |
| Two-phase prediction | edge constraints | small on-device model; send uncertain cases to cloud model |
| Human-in-the-loop | high-stakes or low-confidence cases | route below-threshold predictions to reviewers; use reviews as labels |
| Stateless serving function | scaling | model as pure function of inputs; state in external stores |
| Continued evaluation | silent decay | join predictions with later labels; scheduled evaluation |
| Transform pattern | skew in preprocessing | embed preprocessing in the model/pipeline artifact |
| Model-as-a-service / gateway | many consumers & models | uniform API, auth, routing, logging in front of models (LLM gateway) |

## 21.3 Anti-patterns

| Anti-pattern | Why it hurts |
|---|---|
| Notebook-to-production copy-paste | untested, irreproducible |
| Training and serving code diverge | skew → silent accuracy loss |
| Hard-coded model paths / versions | no safe rollback |
| Fitting preprocessing on all data | leakage, inflated metrics |
| No monitoring 'because it passed tests' | drift goes unnoticed |
| Retraining blindly on schedule without validation | can ship worse models |
| Pickles from untrusted sources | code execution risk |
| One giant model for everything | hard to debug, slow to change |
| Logging raw PII | privacy/compliance breach |

## 21.4 End-to-end reference project (putting the book together)



## 21.5 Production readiness checklist

| ✔ | Item |
|---|---|
| ☐ | Business metric, baseline and acceptance criteria agreed |
| ☐ | Data validated and versioned; lineage recorded |
| ☐ | Experiments tracked; model registered with model card |
| ☐ | Unit, data, model, slice, fairness and integration tests in CI |
| ☐ | Containerised, scanned, secrets externalised |
| ☐ | Deployment strategy with automated rollback |
| ☐ | Latency/throughput load-tested; autoscaling configured |
| ☐ | Monitoring: system, data drift, prediction drift, performance, business; alerts with runbooks |
| ☐ | Retraining policy and champion–challenger process |
| ☐ | Security review, privacy (PII) handling, access control, audit logging |
| ☐ | Explainability available for stakeholders; human oversight where needed |
| ☐ | Cost tracked with budgets (especially LLM apps) |

CHAPTER 22

# Interview Questions & Answers

Fundamentals • tracking & registry • deployment & serving • CI/CD & testing • monitoring & drift • A/B testing & retraining • orchestration & cloud • LLMOps • security, governance & responsible AI • scenarios

## 22.1 Fundamentals

**Q1. What is MLOps?**

Practices and tooling that apply DevOps principles to ML so models can be built, deployed, monitored and improved reliably — versioning code, data and models, automating pipelines, and monitoring statistical behaviour.

**Q2. How is MLOps different from DevOps?**

ML systems depend on data and models as well as code, behave statistically, can degrade without code changes (drift), and need continuous training, data validation and model monitoring in addition to CI/CD.

**Q3. What is continuous training (CT)?**

Automatically retraining and validating models when triggered by schedules, new data, drift or performance drops.

**Q4. Explain MLOps maturity levels.**

Level 0 manual process; level 1 automated training pipeline with continuous training; level 2 automated CI/CD of the pipelines themselves.

**Q5. What is training–serving skew and how do you prevent it?**

A mismatch between features in training and in production. Prevent it with shared feature code or a feature store, preprocessing embedded in the model artifact, and comparing logged serving features with training data.

**Q6. How do you ensure reproducibility?**

Version code (Git), data (DVC/snapshots), environment (Docker, pinned deps), configs; fix seeds; track every run's params, metrics, data version and commit; register models with lineage.

## 22.2 Tracking & registry

**Q7. What does MLflow track?**

Parameters, metrics (including per-step), artifacts (models, plots), tags (git SHA, data version), and run metadata, grouped into experiments.

**Q8. Backend store vs artifact store?**

The backend store holds run metadata, params and metrics (database); the artifact store holds files such as models and plots (S3/Blob/GCS).

**Q9. What is an MLflow flavor and pyfunc?**

A flavor describes how a model library's model is saved/loaded; pyfunc is the generic flavor so any model can be loaded and called with predict() in the same way.

**Q10. Why a model signature?**

It records the input/output schema so serving can validate requests and catch schema mismatches.

### Q11. Experiment tracking vs model registry?

Tracking records every run; the registry manages selected models' versions, aliases, approvals and lifecycle for deployment.

### Q12. Aliases vs stages?

Stages (Staging/Production) were fixed lifecycle states, now deprecated in MLflow; aliases like @champion are flexible pointers to versions, and serving loads by alias so promotion and rollback are alias moves.

### Q13. How do you version data?

DVC or lakeFS (content-hashed pointers in Git, data in remote storage), or immutable dated snapshots / Delta time travel, with the version recorded in each run.

## 22.3 Deployment & serving

### Q14. Online vs batch inference?

Online serves predictions per request with low latency; batch scores large datasets on a schedule cheaply. Choose based on when the prediction is needed.

### Q15. Blue-green vs canary vs shadow?

Blue-green switches all traffic between two identical environments with instant rollback; canary sends a small, increasing share of traffic to the new model; shadow mirrors traffic to the new model without using its outputs.

### Q16. How do you roll back a model?

Point the serving alias/traffic weight back to the previous registered version, ideally automatically when guardrail metrics breach thresholds.

### Q17. Why load the model at startup in FastAPI?

Loading per request adds huge latency and memory churn; loading once in the lifespan handler keeps requests fast.

### Q18. def vs async def for inference endpoints?

CPU-bound model inference should use def (run in a threadpool) or a process pool; async def is for I/O-bound work like calling LLM APIs or databases with async clients.

### Q19. What does TensorFlow Serving provide?

A high-performance server for SavedModels with REST and gRPC APIs, automatic version loading, hot swapping, version labels, and request batching.

### Q20. When use a model server instead of FastAPI?

For high-throughput GPU inference needing dynamic batching, multiple models and versions; FastAPI fits custom business logic and simpler models, often as a gateway in front.

### Q21. How do you reduce inference latency?

Quantisation, distillation, ONNX/TensorRT, batching, caching, right-sized hardware, keeping models warm, and autoscaling.

## 22.4 CI/CD & testing

### Q22. What goes into CI for ML?

Linting, unit tests for feature code, data validation tests, a small training smoke test, and building artifacts.

### Q23. What is a quality gate?

An automated check that blocks promotion unless the model meets absolute thresholds, beats the champion, passes slice and fairness checks and operational limits.

### Q24. How do you test a model?

Performance thresholds, comparison to champion, slice tests, behavioural tests (invariance, directional, minimum functionality), robustness and fairness tests, determinism and serialisation round-trip.

### Q25. What are data validation tests?

Checks on schema, types, ranges, missing values, uniqueness, distributions and leakage, using tools like Pandera or Great Expectations.

### Q26. Why build once and promote the same image?

It guarantees that what was tested in staging is exactly what runs in production.

## 22.5 Monitoring & drift

### Q27. What do you monitor for an ML model?

System metrics, data quality and drift, prediction distribution, model performance when labels arrive, and business KPIs.

### Q28. Data drift vs concept drift?

Data drift is a change in input distribution P(X); concept drift is a change in the relationship P(y|X), which degrades accuracy even if inputs look the same.

### Q29. How is PSI computed and interpreted?

Bin the reference distribution, compute Σ(actual − expected)·ln(actual/expected) across bins; below 0.1 stable, 0.1–0.25 moderate, above 0.25 significant.

### Q30. KS test vs PSI?

KS measures the maximum gap between CDFs with a p-value (very sensitive at large n); PSI is an effect-size style index over bins, popular for thresholds.

### Q31. How do you monitor when labels are delayed?

Use proxies: input drift, prediction drift, confidence, business proxy metrics, and human-labelled samples, then confirm with true labels when they arrive.

### Q32. How do you avoid alert fatigue?

Monitor important features, use effect sizes not only p-values, require persistence across windows, account for seasonality, and attach runbooks.

## 22.6 A/B testing & retraining

### Q33. Why A/B test if offline metrics improved?

Offline metrics may not translate to business impact due to feedback loops, user behaviour, latency and metric mismatch; A/B testing measures causal impact.

### Q34. How do you determine sample size?

From baseline rate, minimum detectable effect, significance level and power using a power analysis.

### Q35. What is peeking and why is it a problem?

Checking results repeatedly and stopping when significant inflates false positives; use fixed horizons or sequential testing.

### Q36. What is sample-ratio mismatch?

When the observed split differs from the intended split, usually indicating an assignment or logging bug that invalidates the test.

### Q37. Bandits vs A/B tests?

Bandits adaptively allocate more traffic to better variants to minimise regret; A/B tests keep fixed splits for cleaner inference.

### Q38. When should you retrain?

On schedule, on performance drops, on significant drift, on enough new labelled data, or on business events — always validated against the champion before promotion.

### Q39. Sliding window vs full history?

Sliding windows adapt faster to change but forget older patterns; full history is stable but slower to adapt; weighted schemes balance both.

### Q40. What are feedback loops in retraining?

When the model's own decisions shape future training data (e.g. recommendations, blocked fraud), biasing it; mitigate with exploration and counterfactual logging.

## 22.7 Orchestration & cloud

### Q41. Why use an orchestrator?

To run multi-step pipelines reliably with dependencies, retries, scheduling, caching, parallelism, logging and lineage.

### Q42. Airflow vs Kubeflow Pipelines?

Airflow is a general workflow scheduler with many operators; Kubeflow Pipelines runs containerised ML components on Kubernetes with artifact tracking and runs natively on Vertex AI.

### Q43. What is a feature store?

A system that defines features once and serves them consistently to training (offline, point-in-time correct) and inference (online, low latency).

### Q44. How do you do canary on SageMaker / Azure ML / Vertex?

SageMaker production variants with weights or deployment guardrails; Azure managed online endpoints with traffic split between deployments; Vertex endpoints with traffic_percentage per deployed model.

### Q45. What is SageMaker script mode?

Running your own training script in a managed framework container, reading data and hyper-parameters from environment/args and saving the model to the model directory.

## 22.8 LLMOps

### Q46. What is LLMOps?

Operating LLM applications: model selection, prompt and configuration versioning, RAG operations, evaluation with golden sets and LLM-as-judge, guardrails, tracing, cost and latency monitoring, and feedback-driven improvement.

### Q47. How do you evaluate an LLM app?

Deterministic checks, a golden dataset, LLM-as-judge rubrics calibrated with humans, RAG metrics (faithfulness, context recall), agent trajectory evals, online feedback and red-teaming.

### Q48. How do you reduce LLM cost?

Route easy requests to smaller models, trim prompts and retrieved context, cap output tokens, use prompt and semantic caching, batch APIs, fewer agent steps, fine-tune small models, or self-host at high volume.

### Q49. What is a semantic cache and its risk?

Reusing answers for semantically similar past queries via embedding similarity; risk of returning a wrong or stale answer, mitigated with strict thresholds, TTLs and scoping.

### Q50. What metrics do you monitor for LLM apps?

Latency and time-to-first-token, tokens and cost, errors and rate limits, feedback and LLM-judge quality scores, groundedness, guardrail triggers, agent steps, and query topic drift.

### Q51. What is prompt injection and how do you defend?

Malicious instructions in user input or retrieved content that hijack the model; defend with least-privilege tools, treating external content as data, classifiers, output checks and human approval for actions.

### Q52. API model vs self-hosted?

APIs give frontier quality with low ops but per-token cost and data-control limits; self-hosting gives control and lower cost at high steady volume but requires GPU operations.

## 22.9 Security, governance & responsible AI

### Q53. How do you handle secrets in ML pipelines?

Store them in a secret manager, inject at runtime with managed identities, never commit them, and scan repositories and images.

### Q54. Why avoid untrusted pickle files?

Unpickling can execute arbitrary code; use trusted sources, signatures/hashes, or safer formats like safetensors/ONNX.

### Q55. What does model governance include?

Model inventory, lineage, documentation (model cards), risk-based approvals, audit trails, monitoring and periodic review.

### Q56. Demographic parity vs equal opportunity?

Demographic parity equalises positive prediction rates across groups; equal opportunity equalises true positive rates among actual positives.

### Q57. How do you mitigate bias?

Pre-processing (reweighing, better data), in-processing (fairness constraints), post-processing (group thresholds), plus monitoring fairness metrics in production.

### Q58. SHAP vs LIME?

SHAP uses Shapley values with consistency and local accuracy guarantees (fast exact TreeSHAP); LIME fits a local linear surrogate on perturbed samples and can be unstable.

### Q59. Global vs local explanations?

Global explains overall model behaviour (feature importance, PDP); local explains a single prediction (SHAP waterfall, LIME, counterfactuals).

## 22.10 Scenarios

### Q60. Your model's accuracy dropped 8% in production but code didn't change. What do you do?

Check data pipeline and schema changes, compare serving vs training feature distributions (drift/skew), check label definitions; if real drift, retrain on recent data, validate vs champion, canary, and add monitors/tests that would have caught it.

### Q61. Design an MLOps pipeline for a churn model at a bank.

Versioned data with validation, Airflow/KFP training pipeline with MLflow tracking, quality and fairness gates, registry with approvals, Docker + FastAPI or managed endpoint, canary release, monitoring of drift/performance/business metrics, scheduled and drift-triggered retraining, model card and audit logs.

### Q62. An LLM chatbot's monthly bill tripled.

Inspect traces for token growth (prompt or context changes, longer histories), agent loops, cache hit rate, traffic changes and model routing; fix with trimming, caching, routing, max tokens and budgets.

### Q63. Stakeholders don't trust the model.

Provide global and local explanations, slice performance, model card, shadow-mode comparison with current process, human-in-the-loop for edge cases, and clear monitoring dashboards.

CHAPTER 23

# Cheat Sheet

Formulas • commands • which tool when

## 23.1 Formulas

| | |
|---|---|
| **PSI** | $\sum (a_i - e_i) \ln(a_i / e_i)$ |
| **KS statistic** | $\sup_x \lvert F_{ref}(x) - F_{cur}(x) \rvert$ |
| **Chi-square** | $\sum (O - E)^2 / E$ |
| **A/B sample size** | $n \approx \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 [p_A(1-p_A) + p_B(1-p_B)]}{(p_B - p_A)^2}$ |
| **Two-proportion z** | $\frac{\hat{p}_B - \hat{p}_A}{\sqrt{\hat{p}(1-\hat{p})(1/n_A + 1/n_B)}}$ |
| **LLM cost** | $(T_{in} p_{in} + T_{out} p_{out}) / 10^6$ |
| **Self-host cost** | $\frac{\$/h}{\text{tok/s} \times 3600} \times 10^6$ |
| **Disparate impact** | $P(\hat{Y} = 1 \mid A = u) / P(\hat{Y} = 1 \mid A = p) \geq 0.8$ |
| **SHAP additivity** | $f(x) = \phi_0 + \sum_j \phi_j$ |

## 23.2 Commands

**commands**

```
mlflow ui | mlflow server --backend-store-uri ... --artifacts-destination ...     # tracking
mlflow models serve -m models:/churn-classifier@champion -p 5001                    # quick serving
dvc add data/train.csv && dvc push | dvc repro                                      # data versioning / pipelines
uvicorn app.main:app --reload | gunicorn app.main:app -k uvicorn.workers.UvicornWorker -w 4
docker build -t api:1.0 . && docker run -p 8000:8000 api:1.0
docker run -p 8501:8501 -v $PWD/models/churn:/models/churn -e MODEL_NAME=churn tensorflow/serving
kubectl apply -f k8s/ | kubectl rollout status deploy/churn-api | kubectl rollout undo deploy/churn-api
pytest -q | locust -f loadtest.py | trivy image api:1.0
```

## 23.3 Which tool when

| Need | Go-to |
|---|---|
| Track experiments | MLflow (or W&B) |
| Version data | DVC / lakeFS / Delta |
| Orchestrate pipelines | Airflow (general), KFP/Vertex Pipelines (K8s/ML), Prefect/Dagster (Python) |
| Serve sklearn/XGBoost | FastAPI + Docker, MLflow serve, BentoML |
| Serve TF/PyTorch at scale | TF Serving, TorchServe, Triton, KServe |
| Serve LLMs | vLLM / TGI or provider APIs via a gateway |
| Validate data | Pandera, Great Expectations |
| Detect drift | Evidently, WhyLogs, cloud model monitors |
| Monitor systems | Prometheus + Grafana, OpenTelemetry |
| Trace LLM apps | LangSmith, Langfuse, Phoenix |
| Explain models | SHAP, LIME, InterpretML |
| Fairness | Fairlearn, AIF360 |
| Managed end-to-end | SageMaker, Azure ML, Vertex AI |

**Watch out:** Code in this guide is reference-quality and was not run in your environment. MLflow, Evidently, Pandera, cloud SDKs and LLM tooling APIs change often — check the documentation for your installed versions. Figures marked 'computed' were generated with NumPy/SciPy/scikit-learn; others are illustrative or synthetic.